

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



Využití open-source a komerčních nástrojů pro
vizualizaci a analýzu dat na datové platformě
Portabo

BAKALÁŘSKÁ PRÁCE

Vypracoval: Ladislav Tahal

Vedoucí práce: Ing. Roman Vaibar, Ph.D., MBA

Studijní program: Aplikovaná informatika

ÚSTÍ NAD LABEM 2025

Děkuji vedoucímu bakalářské práce Ing. Romanu Vaibarovi, Ph.D., MBA
za jeho odborné vedení a praktické podněty, které výrazně přispěly k analýze dat,
hledání efektivních řešení a k celkovému dokončení bakalářské práce.

Abstrakt:

Bakalářská práce se zabývá srovnáním open-source a komerčních nástrojů pro vizualizaci a analýzu dat, datové sklady a OLAP technologie s cílem zhodnotit jejich vhodnost pro využití v datové platformě Portabo. V praktické části byla realizována implementace několika variant datových řešení, zahrnujících kombinace databázových systémů MSSQL, MariaDB, ClickHouse a PostgreSQL s nástroji Power BI, Superset a Cube.js. Bylo provedeno testování výkonu, analýza technických požadavků, nároků na uživatelské dovednosti a srovnání ekonomických aspektů provozu. Výsledkem práce je komplexní přehled výhod a nevýhod jednotlivých přístupů, měření jejich efektivity při zpracování velkých objemů dat a doporučení optimálního řešení pro organizace usilující o efektivní datovou analytiku bez nutnosti vysoké IT expertizy.

Klíčová slova: Business Intelligence, datové sklady, OLAP, vizualizace dat, Portabo

UTILIZATION OF OPEN-SOURCE AND COMMERCIAL TOOLS FOR DATA VISUALIZATION AND ANALYSIS ON THE PORTABO DATA PLATFORM

Abstract:

This bachelor's thesis focuses on the comparison of open-source and commercial tools for data visualization and analysis, data warehouses, and OLAP technologies, with the aim of evaluating their applicability within the Portabo data platform. In the practical part, several data architecture variants were implemented, combining database systems such as MSSQL, MariaDB, ClickHouse, and PostgreSQL with visualization tools including Power BI, Superset, and Cube.js. The analysis covers performance testing, technical requirements, user skill demands, and economic aspects of operation. The outcome of the thesis is a comprehensive overview of the advantages and limitations of both open-source and commercial solutions, performance evaluation on large data volumes, and recommendations for organizations seeking efficient data analytics without requiring extensive IT expertise.

Keywords: Business Intelligence, data warehouses, OLAP, data visualization, Portabo

Obsah

Úvod	13
1. Přehled současného stavu problematiky	15
1.1. Rešerše v oblasti datových skladů, OLAP technologií a nástrojů pro vizualizaci dat	15
1.2. Přehled současných open-source a komerčních řešení	16
1.3. Analýza trendů v implementaci	24
2. Teoretická část	29
2.1. Úvod do problematiky datových skladů a OLAP	29
2.2. Nástroje Business Intelligence	35
2.3. Přehled a charakteristika využitých technologií	37
3. Praktická část	41
3.1. Návrh metodiky porovnání	44
3.2. Příprava dat a návrh datového skladu	44
3.3. Provedení srovnávací analýzy	54
3.4. Zhodnocení výsledků a doporučení	68
4. Závěr	71
A. Externí přílohy	77
B. Testovací SQL dotazy	79
Seznam použitých zkratk a pojmů	83

Úvod

V současné době organizace generují a shromažďují obrovské množství dat, která se stávají klíčovým zdrojem pro strategické i operativní rozhodování. Společnosti napříč odvětvími se proto zaměřují na to, jak tato data efektivně zpracovávat, ukládat, analyzovat a vizualizovat tak, aby přinášela skutečnou informační hodnotu i uživatelům bez hlubokého technického zázemí. S rozvojem datových technologií vzniká široké spektrum nástrojů, které umožňují tvorbu reportů, analytických modelů a vizualizací nad velkými objemy dat. Tyto nástroje mohou být jak komerční, nabízející komplexní podporu a integrované služby, tak open-source, které poskytují flexibilitu a otevřenost.

Zároveň však s tímto rozvojem vyvstává otázka, jaké řešení je pro konkrétní organizaci nejvhodnější – z pohledu technického, uživatelského i ekonomického. Volba správného nástroje či architektury datové platformy zásadně ovlivňuje nejen efektivitu zpracování dat, ale také dostupnost a interpretaci výsledků pro koncové uživatele.

Tato bakalářská práce se zabývá využitím open-source a komerčních nástrojů pro vizualizaci a analýzu dat na datové platformě Portabo. Cílem práce je provést srovnávací analýzu vybraných řešení z hlediska technických parametrů, uživatelských požadavků a ekonomických aspektů jejich provozu. Praktická část je zaměřena na implementaci několika variant datové architektury – zahrnujících kombinace databázových systémů MSSQL, MariaDB, ClickHouse a PostgreSQL s nástroji Power BI, Superset a Cube.js. Součástí analýzy je také měření výkonu při práci s rozsáhlým datovým souborem, testování zátěže a vyhodnocení celkových nákladů na provoz (TCO).

První kapitola práce shrnuje současný stav problematiky, základní pojmy z oblasti datových skladů, OLAP, vizualizačních nástrojů a aktuální trendy v oblasti datové analytiky. Následující teoretická část rozebírá principy a architekturu zvolených technologií. Praktická část popisuje návrh metodiky, implementaci jednotlivých řešení a provedení testování. Závěrečná kapitola obsahuje shrnutí zjištěných výsledků, jejich interpretaci a doporučení vhodného řešení.

1. Přehled současného stavu problematiky

1.1. Rešerše v oblasti datových skladů, OLAP technologií a nástrojů pro vizualizaci dat

Tato kapitola přináší přehled současného stavu v oblasti zpracování, ukládání a vizualizace dat. Cílem je zmapovat, jak jsou dnes řešeny datové platformy určené pro analytické účely, jaké nástroje se používají v praxi a jaké trendy určují směr vývoje v oblasti datové analytiky.

Datové sklady a vývoj analytických databází

Datové sklady a technologie OLAP patří k dlouhodobě stabilním základům datové analytiky. Již několik desetiletí tvoří klíčovou infrastrukturu pro zpracování a konsolidaci dat z různých zdrojů, přičemž jejich architektura se neustále vyvíjí s ohledem na rostoucí objem dat a potřebu vyšší výpočetní efektivity.

Původně byly analytické systémy budovány nad relačními databázemi. Tyto systémy dominovaly zejména podnikovému prostředí a poskytovaly základní podporu pro tvorbu datových skladů. Postupně se však začala prosazovat specializovaná řešení určená přímo pro analytické zpracování velkých objemů dat.

Moderní analytické databáze, jako jsou *ClickHouse*, *Snowflake*, *Amazon Redshift* či *Google BigQuery*, využívají kolumnární uložení dat, paralelní zpracování a škálovatelnou cloudovou architekturu [1, 2]. Výhodou těchto systémů je možnost provádět rozsáhlé analytické dotazy v reálném čase a minimalizovat potřebu předběžných agregací. Vedle komerčních řešení se rozvíjí i open-source alternativy, které kombinují vysoký výkon s flexibilitou nasazení a nezávislostí na dodavateli – například *ClickHouse* či *Apache Druid*.

OLAP technologie a analytické přístupy

OLAP technologie zůstávají jedním z hlavních způsobů, jak efektivně analyzovat a agregovat data pro potřeby rozhodování. Tradiční přístupy založené na předpočítaných datových kostkách jsou dnes doplňovány flexibilnějšími modely, které umožňují ad-hoc analýzy nad velkými datovými sadami.

V současnosti lze pozorovat trend integrace OLAP funkcionality přímo do datových skladů. Například platformy jako *Snowflake*, *ClickHouse* nebo *PostgreSQL* s rozšířením *TimescaleDB* umožňují analytické dotazování bez nutnosti samostatné OLAP vrstvy [1]. Výsledkem je zjednodušená architektura a nižší náklady na údržbu.

OLAP se zároveň posouvá směrem k real-time zpracování, kde je možné kombinovat streamovaná a historická data. Tento přístup umožňuje okamžitou reakci na události, což je zásadní například v oblasti IoT, průmyslového monitoringu nebo dopravní analýzy.

Nástroje pro vizualizaci a business intelligence

Vizualizační a BI nástroje představují prezentační vrstvu datové analytiky. Umožňují nejen tvorbu reportů a dashboardů, ale i interaktivní exploraci dat bez nutnosti pokročilých programátorských znalostí. Nároky na uživatele se liší v závislosti na architektuře řešení. V případě přímého napojení na databázi může být vyžadována znalost jazyka SQL. Pokud je však využita sémantická vrstva, uživatel pracuje již s předpřipravenými datovými objekty, které pouze vizualizuje. Pro pokročilejší úpravy či specifické kalkulace lze následně využít specializované jazyky daného nástroje, jako jsou například DAX pro výpočty nebo Power Query pro transformaci dat v prostředí Power BI. Na trhu existuje široké spektrum nástrojů, od komerčních platforem po open-source řešení. Mezi nejpoužívanější komerční systémy patří *Microsoft Power BI*, *Tableau* a *Qlik Sense*. Tyto produkty nabízejí komplexní ekosystém pro tvorbu vizualizací, datových modelů a propojení s různými datovými zdroji.

Z open-source nástrojů se výrazně prosadily *Apache Superset*, *Grafana* a *Metabase*, které poskytují flexibilní možnosti přizpůsobení, automatizace a integrace. V této oblasti se dynamicky rozvíjí také samostatné nástroje pro tvorbu *sémantické vrstvy* (semantic layer). Ta slouží jako překladový můstek mezi technickou strukturou databáze a byznysovým pohledem uživatele. Reprezentantem tohoto přístupu je například framework *Cube.js*, který umožňuje firmám zpřístupnit datový sklad i uživatelům bez hluboké znalosti SQL [3, 4, 5].

1.2. Přehled současných open-source a komerčních řešení

V současné době existuje řada komplexních ekosystémů určených pro správu, zpracování a analýzu dat. Tyto ekosystémy zpravidla zahrnují nástroje pro integraci dat (ETL/ELT), datové sklady, OLAP

vrstvy a vizualizační rozhraní. Cílem této části je poskytnout rešeršní přehled nejrozšířenějších řešení, která se využívají při budování datových platforem.

Před samotným přehledem je důležité vymezit klíčové pojmy, které budou v práci dále používány. Jedná se především o dělení na **komerční** a **open-source** software a dále na **on-premise** a **cloudová** řešení.

Klasifikace řešení

- **Komerční software** je software, jehož distribuce a používání je podmíněno zakoupením licence. Jeho *zdrojový kód je uzavřený (proprietární)*, což znamená, že uživatelé nemají kód k dispozici, nemohou jej prohlížet, modifikovat, ani provádět zpětné inženýrství. Důsledkem je *nemožnost modifikovat* chování systému a *potenciální závislost na dodavateli* (tzv. *vendor lock-in*) v otázkách úprav a oprav. Komerční řešení bývají vyvíjena a distribuována za účelem zisku. Výhodou bývá profesionální podpora, garantovaná kvalita a ucelený ekosystém nástrojů.
- **Open-source software** [6] poskytuje uživatelům přístup k veřejně dostupnému zdrojovému kódu a uděluje jim práva k jeho používání, studování, modifikaci a šíření. Klíčovým prvkem je zde *open-source licence*, která přesně definuje, jakým způsobem lze s dílem a jeho modifikacemi nakládat. *Detailní pochopení licenčních podmínek je pro praktické nasazení zásadní* (pro přehled a podmínky licencí je nutné nahlédnout do oficiální dokumentace konkrétní licence).
 - *Permisivní licence*¹ (např. MIT, Apache 2.0) jsou maximálně benevolentní, kladou minimální omezení a obvykle umožňují upravený kód uzavřít a následně komerčně prodávat jako proprietární produkt.
 - *Copyleftové licence*² (např. GPL) jsou přísnější a vyžadují, aby jakékoliv odvozené dílo nebo úprava byly šířeny pod stejnou svobodnou licencí. Tím je zajištěno, že zdrojový kód zůstane trvale veřejně přístupný i v budoucích verzích.

Mezi obecné výhody open-source softwaru patří nezávislost na dodavateli, nižší počáteční náklady a možnost bezpečnostního auditu komunitou. Nevýhodou mohou být vyšší nároky na technické znalosti při implementaci a nutnost řešit podporu buď komunitně, nebo placenou službou třetí strany.

- **On-premise řešení** je provozováno na vlastní infrastruktuře organizace (vlastních serverech). To poskytuje plnou kontrolu nad daty a systémem, ale zároveň vyžaduje vyšší počáteční investice do hardwaru a personálu pro jeho správu [7].
- **Cloudové řešení** je poskytováno jako služba třetí stranou. Podle úrovně správy, kterou přebírá poskytovatel, rozlišujeme tři základní modely:

¹Vysvětlení permisivních licencí: <https://opensource.org/licenses>

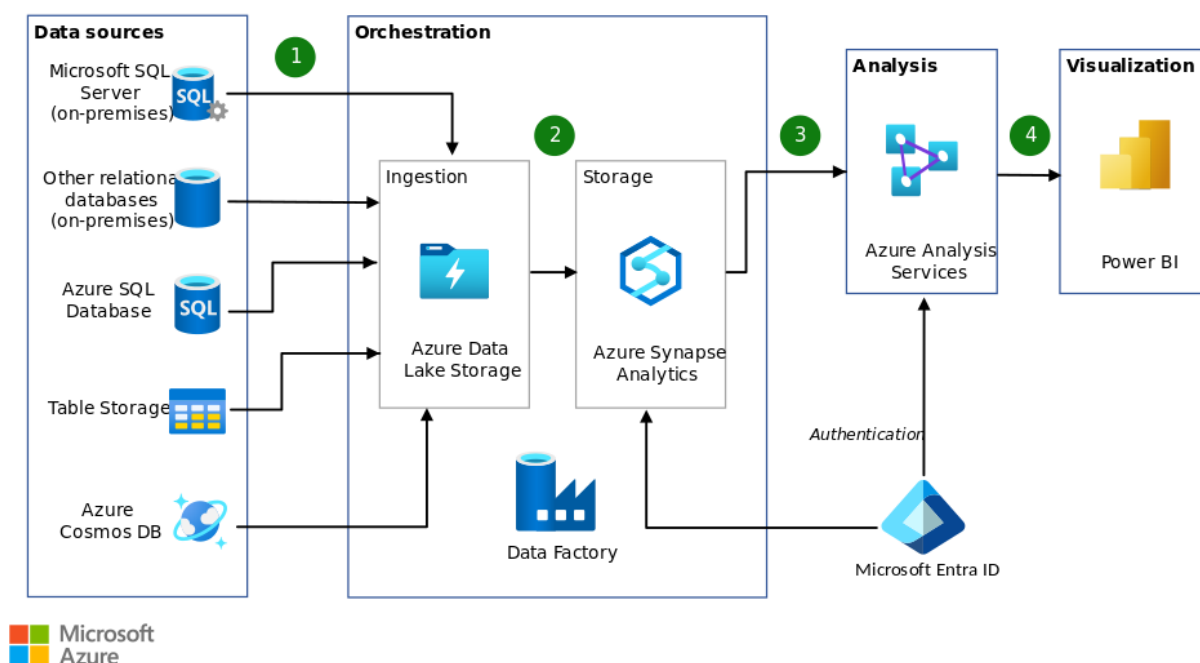
²Vysvětlení copyleftových licencí: <https://www.gnu.org/licenses/copyleft.en.html>

- **IaaS:** Poskytovatel zajišťuje hardware, virtualizaci a síť. Uživatel spravuje operační systém a aplikace (např. Amazon EC2).
- **PaaS:** Poskytovatel spravuje i operační systém a runtime prostředí. Uživatel se stará pouze o aplikace a data (např. Google App Engine).
- **SaaS:** Kompletní softwarové řešení spravované poskytovatelem. Uživatel pouze využívá službu (např. Google Workspace, Microsoft 365).

Výhodou je škálovatelnost, platba za skutečné využití (pay-as-you-go) a menší nároky na vlastní IT oddělení. Nevýhodou může být menší kontrola nad infrastrukturou a potenciální závislost na poskytovateli.

Tato klasifikace je zásadní pro pozdější srovnání celkových nákladů na vlastnictví (TCO), kde se projeví rozdíly v licenčních poplatcích, nákladech na infrastrukturu a správu.

Microsoft Data Platform (komerční, on-premise/cloud)



Obrázek 1.1.: Architektura datového skladu v prostředí Microsoft Azure. Zdroj: learn.microsoft.com [8]

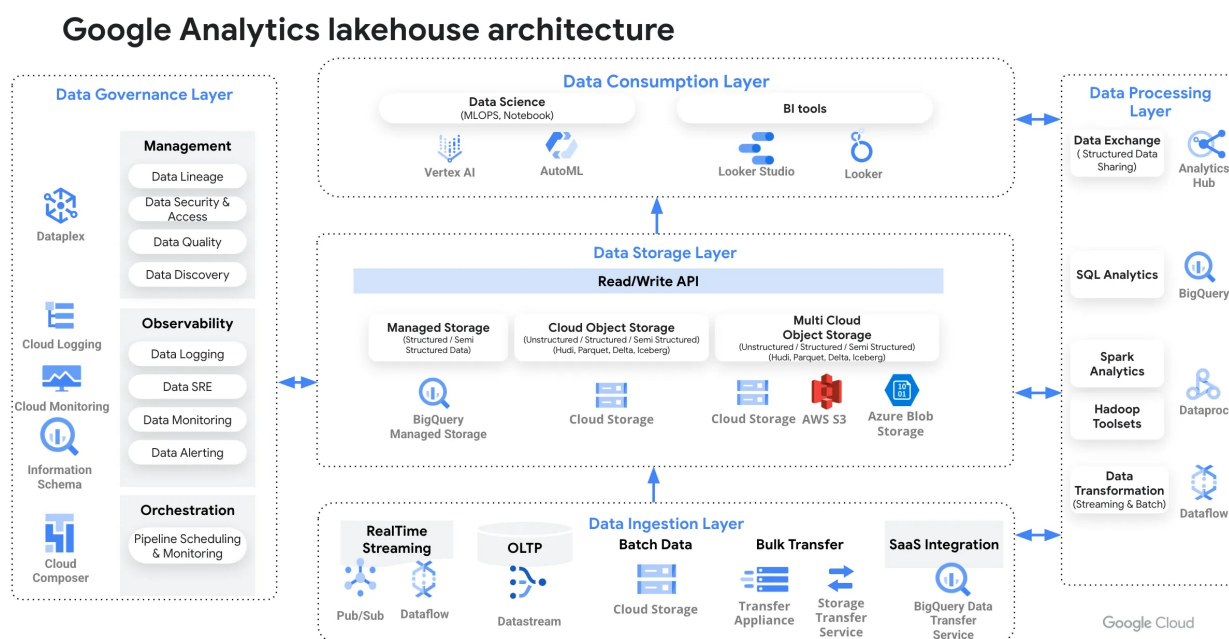
Ekosystém společnosti Microsoft představuje jedno z nejkomplexnějších řešení pro podnikové zpracování dat, které pokrývá celý životní cyklus od datových toků až po vizualizaci. Jeho unikátnost spočívá v možnosti nasazení jak v tradičním on-premise prostředí, tak v cloudu, případně v hybridním režimu. Komponenty platformy lze rozdělit následovně:

- **Tradiční on-premise infrastruktura:** Základem je *Microsoft SQL Server*. Pro integraci dat (ETL) je využíván modul *SQL Server Integration Services (SSIS)* a pro analytické modelování

(OLAP) slouží *SQL Server Analysis Services (SSAS)*. Toto složení tvoří osvědčený standard pro lokální datové sklady ³.

- **Cloudová platforma (Microsoft Azure):** S nástupem cloudu se architektura posouvá do prostředí Azure (viz obrázek 1.1). Roli datového skladu zde přebírá *Azure Synapse Analytics*, ETL procesy zajišťuje *Azure Data Factory* a celková orchestrace se sjednocuje v rámci platformy *Microsoft Fabric*.
- **Vizuální a sémantická vrstva:** Sjednocující vrstvou pro on-premise i cloud je *Power BI*. Tento nástroj slouží pro tvorbu reportů a dashboardů a díky své sémantické vrstvě dokáže zastoupit i některé funkce SSAS ⁴.
- **Výhody a nevýhody:** Hlavní výhodou je hluboká integrace všech komponent. Nevýhodou, zejména u cloudových služeb, může být silný „vendor lock-in“ a závislost na licenční politice poskytovatele.

Google Cloud Data Analytics (komerční, cloud)



Obrázek 1.2.: Architektura analytické platformy Google Cloud. Zdroj: medium.com [9]

Společnost Google přistupuje k analytice jako k plně cloudové službě, navržené od počátku pro distribuované výpočty. Klíčové komponenty ekosystému jsou:

- **Datový sklad:** Jádrem platformy je *Google BigQuery*, serverless datový sklad s kolumnárním uložením, kde se platí pouze za uložená data a zpracované dotazy ⁵.

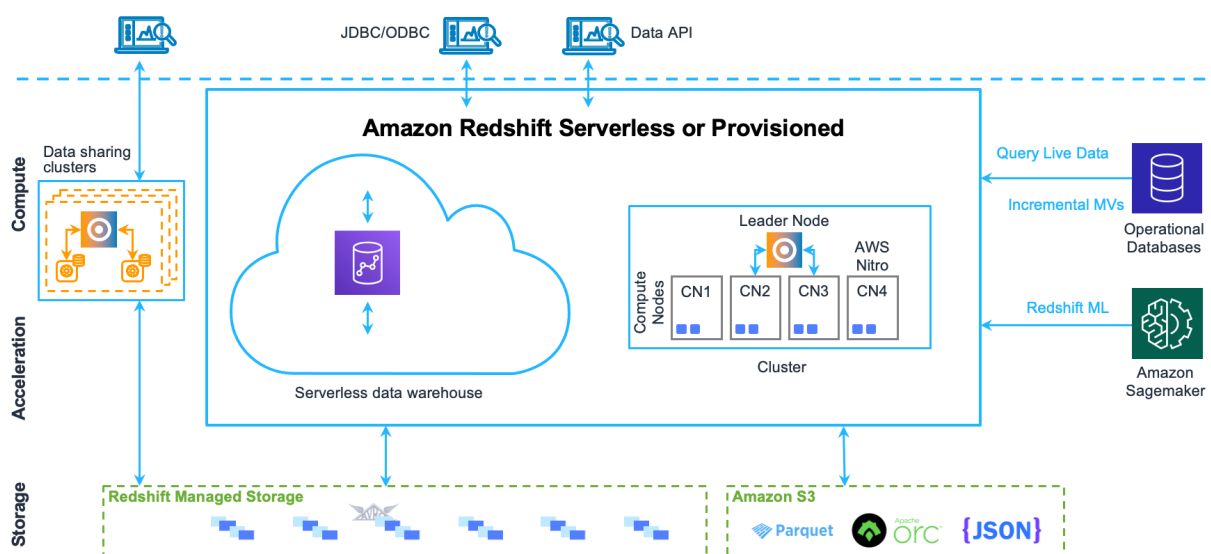
³<https://learn.microsoft.com/en-us/sql/sql-server/>

⁴<https://learn.microsoft.com/en-us/power-bi/>

⁵<https://cloud.google.com/bigquery/docs/>

- **Integrace a příprava dat:** Pro tyto účely slouží služby jako *Dataflow* (založené na Apache Beam) nebo *Dataprep*.
- **Prezentační vrstva:** Zastupuje ji *Looker Studio* pro rychlou vizualizaci, případně platforma *Looker*, která nabízí pokročilou sémantickou vrstvu a governance dat.
- **Výhody a omezení:** Hlavní výhodou je nulová starost o hardware, vysoká škálovatelnost a integrace s AI nástroji (*Vertex AI*). Zásadním omezením je absence on-premise řešení, což znemožňuje využití organizací, které vyžadují uložení dat na vlastním hardwaru.

Amazon Web Services (komerční, cloud)



Obrázek 1.3.: Cloudové řešení Amazon Web Services. Zdroj: docs.aws.amazon.com [10]

Cloudová platforma *Amazon Web Services* (AWS) od společnosti Amazon nabízí rozsáhlý a modulární ekosystém pro datové inženýrství a analytiku.

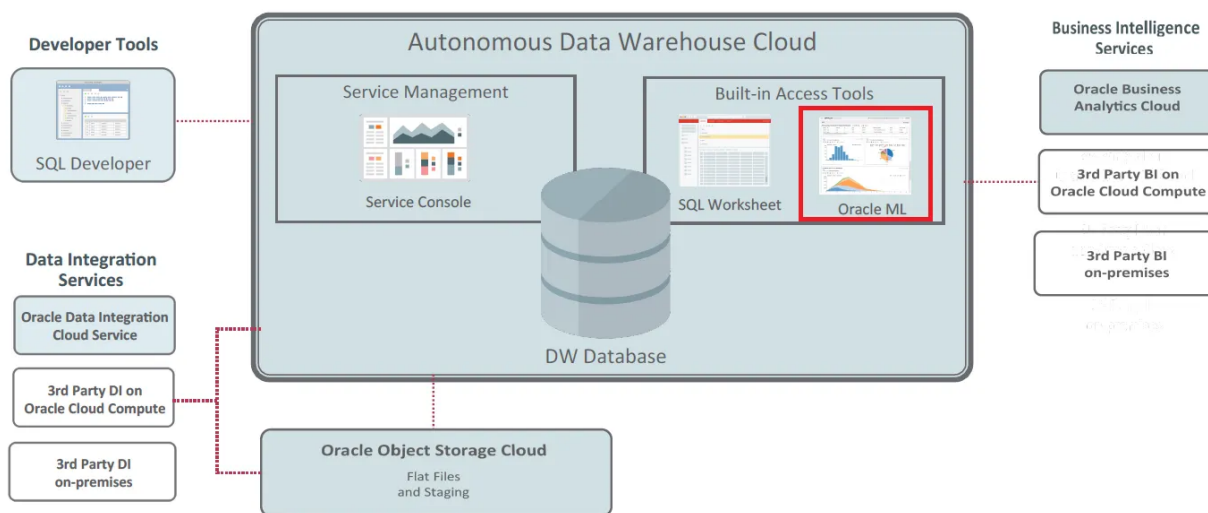
- **Datový sklad:** Klíčovou roli zde zastává *Amazon Redshift* – cloudový datový sklad postavený na kolumnární architektuře ⁶.
- **Zpracování a vizualizace:** Služba *AWS Glue* umožňuje automatizované ETL procesy, zatímco *Amazon QuickSight* poskytuje prostředí pro interaktivní vizualizace a reporting.
- **Integrace s open-source:** AWS podporuje integraci s open-source systémy, například *Apache Spark* (prostřednictvím služby *EMR*) nebo *Presto*.
- **Výhody a slabiny:** Výhodou AWS je modularita, která umožňuje komponenty kombinovat podle potřeb. Podobně jako u Google se jedná o výhradně cloudové řešení. Slabinou může být větší komplexita nastavení a vyšší provozní náklady.

⁶<https://aws.amazon.com/redshift/>

Oracle Data Platform (komerční, cloud/on-premise)

Společnost Oracle nabízí komplexní datovou platformu, která kombinuje tradiční silné stránky v oblasti relačních databází s moderními cloudovými technologiemi. Její ekosystém je navržen tak, aby pokrýval celý životní cyklus dat, od jejich pořízení až po pokročilou analytiku a využití umělé inteligence.

Architecture

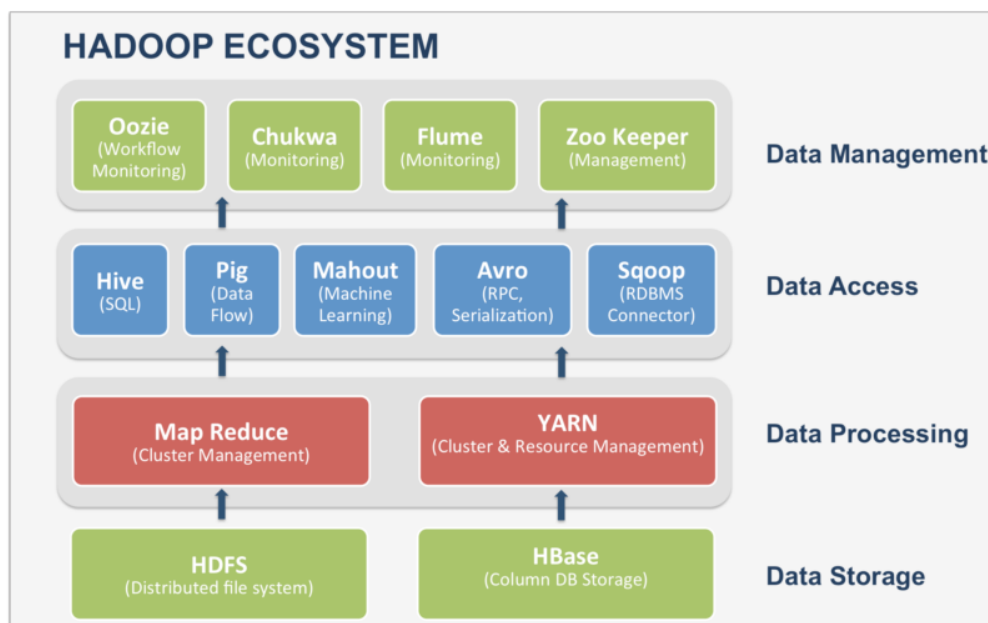


Obrázek 1.4.: Architektura Oracle Data Platform. Zdroj: medium.com [11]

- **Datový sklad a databáze:** Vlajkovou lodí je *Oracle Autonomous Data Warehouse* (ADW), který využívá strojové učení k automatizaci správy, ladění výkonu a zabezpečení. Pro transakční a smíšené zátěže je k dispozici *Oracle Database*, kterou lze provozovat jak v cloudu (OCI), tak na on-premise hardware.
- **Integrace a správa dat:** Služba *Oracle Cloud Infrastructure (OCI) Data Integration* zajišťuje bezkódové ETL/ELT procesy. Pro správu metadat a governance slouží *OCI Data Catalog*.
- **Analytika a vizualizace:** *Oracle Analytics Cloud* (OAC) poskytuje nástroje pro samoobslužnou vizualizaci, reporting a rozšířenou analytiku s podporou AI.
- **Výhody a nevýhody:** Hlavní předností je vysoký výkon, bezpečnost a možnost hybridního nasazení (Cloud@Customer), což oceňují zejména velké korporace a státní správa. Nevýhodou mohou být vyšší licenční náklady a komplexita celého ekosystému.

Ekosystém Apache Software Foundation

Nejvýznamnějším hráčem v oblasti otevřených datových technologií je nadace *Apache Software Foundation* (ASF). Její ekosystém de facto definoval standardy pro zpracování velkých dat (Big Data). Produkty pod hlavičkou Apache lze skládat do výkonné platformy pokrývající celý datový cyklus:



Obrázek 1.5.: Cloudové řešení Apache Software Foundation. Zdroj: blogs.perficient.com

- **Ukládání a zpracování:** Historicky založeno na ekosystému *Hadoop*, dnes dominuje *Apache Spark* pro distribuované výpočty a *Apache Kafka* pro streamování dat.
- **OLAP a analytika:** Pro rychlé analytické dotazy slouží nástroje jako *Apache Druid* nebo *Apache Kylin*, které přinášejí schopnost multidimenzionální analýzy (OLAP kostky) na velká data, podobně jako SSAS v ekosystému Microsoft.
- **Vizualizace:** Prezentační vrstvu zajišťuje *Apache Superset*, moderní BI nástroj umožňující tvorbu dashboardů a granularní řízení přístupových práv.

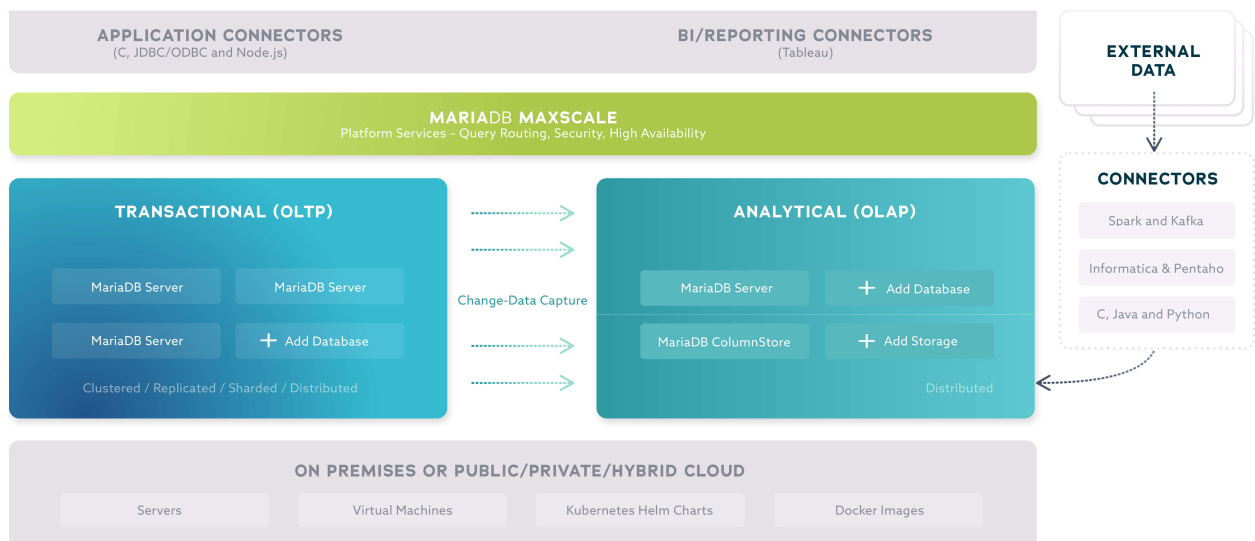
Výhodou tohoto ekosystému je jeho modularita a fakt, že tvoří technologický základ i pro mnoho komerčních cloudových služeb (např. AWS EMR).

Platforma MariaDB (Community / Enterprise)

Příkladem open-source ekosystému, který se transformoval do podoby robustní podnikové platformy, je *MariaDB*. Ačkoliv původně vznikla jako fork databáze MySQL, ve své edici *MariaDB Enterprise Platform* dnes nabízí sadu integrovaných komponent pro transakční, analytické i hybridní zpracování dat a nově i pro umělou inteligenci.

Základem platformy je *MariaDB Enterprise Server*, který podporuje práci s relačními daty i formátem JSON. Síla ekosystému však spočívá v jeho specializovaných modulech a službách:

- **Analytika a Big Data:** Pro analytické úlohy slouží *MariaDB ColumnStore*, což je sloupcové úložiště umožňující rychlé agregace bez nutnosti vytvářet složité indexy. Novinkou je komponenta *MariaDB Exa*, in-memory MPP (Massively Parallel Processing) databáze navržená pro extrémně rychlé SQL dotazy nad velkými objemy dat. Propojení transakčního a analytického světa zajišťuje funkce *Query Accelerator*, která automaticky deleguje náročné dotazy do ColumnStore engine.



Obrázek 1.6.: Architektura řešení MariaDB. Zdroj: smartdatainstitute.com

- **Vysoká dostupnost a správa:** Klíčovým prvkem infrastruktury je *MariaDB MaxScale*. Jde o inteligentní proxy server, který zajišťuje load balancing, zabezpečení a automatické failover procesy. Pro orchestraci a monitorování celé flotily databází slouží *MariaDB Enterprise Manager*.

Tato architektura umožňuje organizacím začít s open-source řešením (Community verze) a v případě potřeby škálovat na plnohodnotnou enterprise platformu s podporou pro AI a real-time analytiku, aniž by musely migrovat na proprietární databázové systémy typu Oracle či MS SQL.

Moderní modulární a hybridní architektury

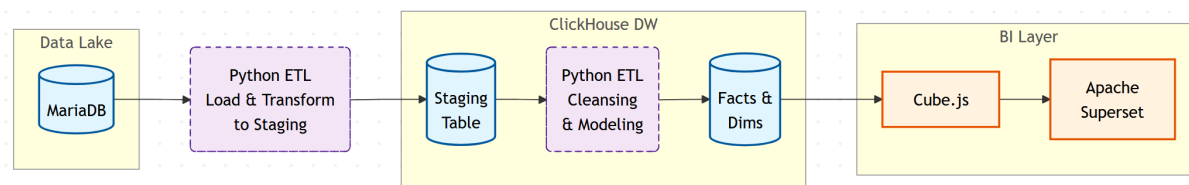
V současné praxi se stále častěji ustupuje od využívání jednoho monolitického „megabalíku“ (jako je kompletní Apache stack) směrem k flexibilním, modulárním architektuрам. Tento trend umožňuje skládat platformu z několika specializovaných nástrojů od různých tvůrců, a to přesně podle potřeb daného projektu [2]. Tento přístup lze realizovat dvěma hlavními způsoby:

- **Čistě open-source stack:** Tento model kombinuje výhradně open-source komponenty. Typickým příkladem je použití relační databáze (např. *MariaDB*, *PostgreSQL*, či *ClickHouse*) jako datového skladu, nad kterým je postavena sémantická vrstva v podobě *Cube.js* nebo *dbt Semantic Layer*⁷. Tyto nástroje definují metriky a business logiku, kterou následně konzumuje vizualizační nástroj jako *Apache Superset*⁸, *Metabase*⁹ nebo *Grafana* [3].
- **Hybridní stack:** V praxi je velmi běžné kombinovat open-source a komerční technologie, aby se využily výhody obou světů. Častým scénářem je provozování výkonné a nákladově efektivní open-source databáze (např. *MariaDB*, *PostgreSQL*) jako datového skladu, nad nímž je nasazen uživatelsky přívětivý komerční vizualizační nástroj jako *Power BI* nebo

⁷<https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>

⁸<https://superset.apache.org/docs/>

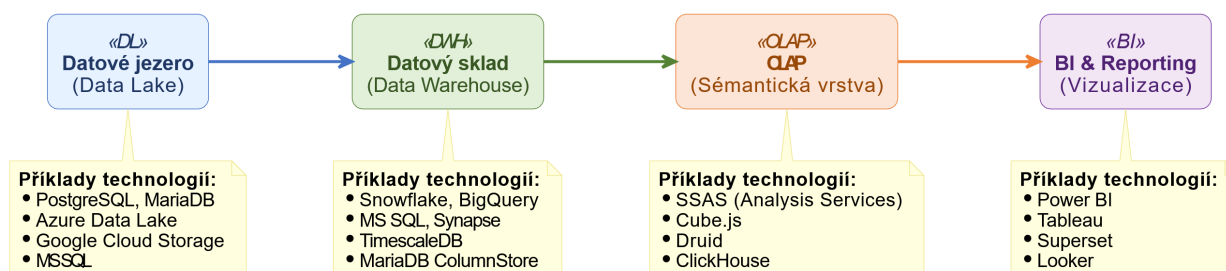
⁹<https://www.metabase.com/>



Obrázek 1.7.: Příklad možné plně open-source sestavy - Zdroj: vlastní zpracování pomocí aplikace Mermaid Live Editor [12]

Tableau. Tento přístup umožňuje optimalizovat náklady na back-end a zároveň poskytnout koncovým uživatelům komfort a pokročilé funkce komerčních platforem.

Hlavní výhodou modulárních přístupů je vysoká míra škálovatelnosti, transparentnost a nezávislost na jednom dodavateli, což umožňuje reagovat na technologické změny a optimalizovat náklady. Společnou slabinou je však vyšší technická složitost. Integrace komponent z odlišných prostředí vyžaduje nejen znalost různých technologií, ale i pochopení jejich licenčních modelů a zajištění vzájemné kompatibility. Pro úspěšnou implementaci je proto klíčová standardizace rozhraní a pečlivé řízení verzí jednotlivých nástrojů.



Obrázek 1.8.: Příklad možného hybridního systému. Zdroj: vlastní zpracování pomocí aplikace PlantUML Editor [13]

1.3. Analýza trendů v implementaci

V posledních letech dochází v oblasti implementace datových platforem a systémů pro analýzu dat k významným změnám. Tyto změny jsou důsledkem rostoucích požadavků na rychlost zpracování, automatizaci a dostupnost analytických nástrojů i mimo oblast IT. Současný vývoj ukazuje posun od uzavřených, monolitických řešení směrem k otevřeným, modulárním a škálovatelným architekturám, které umožňují flexibilnější integraci technologií a snadnější rozšiřování podle potřeb organizace [14, 15].

Automatizace a ELT přístup

Jedním z hlavních trendů posledních let je přechod od tradičního přístupu *ETL* (Extract–Transform–Load) k modernějšímu konceptu *ELT* (Extract–Load–Transform). Transformace dat se tak přesouvá z úrovně samostatných procesů do samotného datového skladu, který disponuje dostatečným výkonem pro jejich zpracování. Tento přístup zjednodušuje datové toky, zvyšuje jejich transparentnost a umožňuje verzování datových modelů. Rozšířené je využití nástrojů jako *dbt*, *Apache Airflow* nebo *Fivetran*, které podporují automatizované a reprodukovatelné datové pipeline.

Datová governance a kvalita dat

S rostoucím objemem dat získává na významu jejich správa, kvalita a dohledatelnost. Moderní přístupy kladou důraz na principy *data governance*, které zahrnují řízení přístupových práv, dokumentaci datových toků (*data lineage*) a sledování kvality dat. K rozšířeným nástrojům patří například *Apache Atlas*, *Amundsen* či *DataHub*, které umožňují centralizovanou správu metadat. V komerční sféře pak obdobné funkce zajišťují systémy jako *Microsoft Purview* nebo *Collibra*. Implementace těchto principů je klíčová nejen z hlediska efektivity, ale i souladu s legislativními požadavky, například GDPR nebo ISO 27001.

Dotazování přirozeným jazykem a role LLM

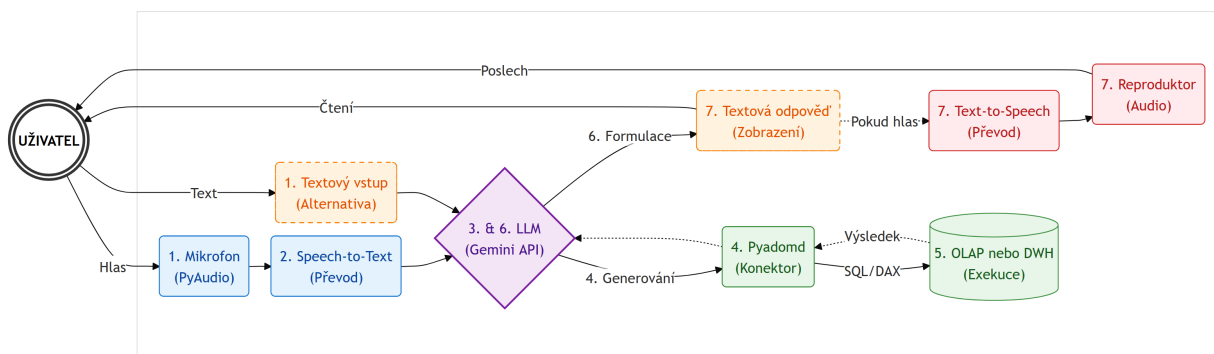
V tomto roce lze pozorovat výrazný rozvoj velkých jazykových modelů (LLM, *Large Language Models*) a jejich integraci do analytických platforem. Tento trend, často označovaný jako *Natural Language Querying* (NLQ), slibuje revoluci ve způsobu, jakým uživatelé interagují s daty. Místo psaní složitých SQL dotazů nebo manuálního proklikávání dashboardů mohou uživatelé pokládat otázky v přirozeném jazyce, a to jak v psané formě (zadáním textu), tak i hlasem.

Architektura řešení

Systém pro dotazování přirozeným jazykem (NLQ) představuje komplexní architekturu složenou z několika komponent, které spolupracují na transformaci lidské řeči či psaného textu na strojově čitelný dotaz a následně zpět na srozumitelnou odpověď.

Níže popsaná architektura vychází z návrhu, který autor implementoval v praxi v rámci firemního prostředí. Toto řešení vzniklo jako reakce na limity standardních dashboardů v nástroji Power BI, konkrétně na časovou náročnost (latenci) při získávání specifických ad-hoc informací. Navržený koncept kombinuje hlasové i textové rozhraní s analytickým backendem a koresponduje s aktuálními trendy v oblasti moderní Business Intelligence.

Typický proces zpracování dotazu lze rozdělit do následujících kroků:



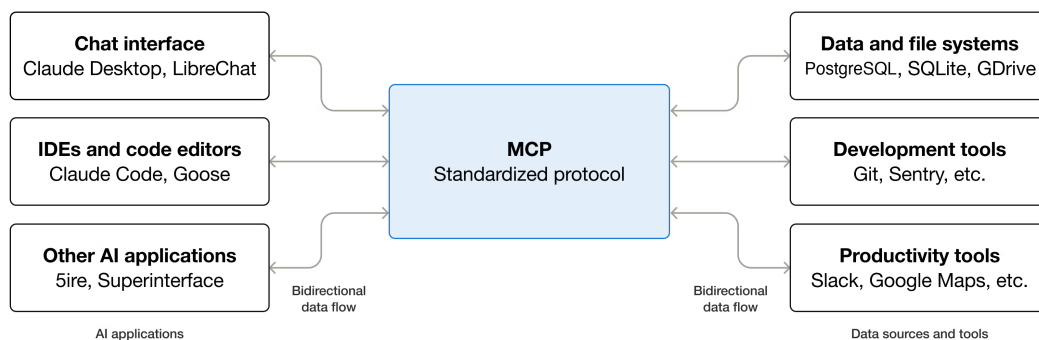
Obrázek 1.9.: Návrh NLQ architektury. Zdroj: vlastní zpracování

- Zpracování hlasového vstupu:** Proces začíná položením dotazu uživatelem, například: „Zjisti, kolik tun papíru bylo vyrobeno papírenským strojem 10 za poslední týden.“ Tento zvukový vstup je zachycen mikrofonom a technicky zpracován pomocí audio knihovny, například PyAudio.
- Převod řeči na text (Speech-to-Text) / Příjem textového vstupu:** Pokud byl vstup hlasový, zvukový záznam je odeslán do služby pro rozpoznávání řeči, která zajistí jeho transkripci do textové podoby. Pokud byl vstup textový, dojde k jeho přímému předání k dalšímu zpracování.
- Sémantická analýza a interpretace (LLM):** Získaný text je předán velkému jazykovému modelu (např. prostřednictvím Gemini API). Model provede sémantickou analýzu, identifikuje záměr uživatele (intent) a extrahuje klíčové entity, jako jsou měřené metriky („tuny papíru“), časové filtry („poslední týden“) a dimenze („papírenský stroj 10“).
- Generování dotazu a orchestrace:** Na základě interpretace LLM vygeneruje odpovídající databázový dotaz. V tomto kroku model nepracuje s daty přímo, ale funguje jako orchestrátor, který připraví syntaxi pro externí nástroj.
- Exekuce dotazu:** Vygenerovaný požadavek (typicky v jazyce DAX nebo SQL) je vykonán nad cílovým datovým skladem nebo OLAP kostkou. Pro komunikaci s technologiemi jako MS SSAS lze využít specializované knihovny, např. Pyadomd.
- Zpracování výsledku a formulace odpovědi:** Číselný výsledek dotazu (např. hodnota 420) je vrácen jazykovému modelu. Ten jej zasadí do kontextu a zformuluje přirozenou větu: „Za poslední týden bylo vyrobeno 420 tun papíru papírenským strojem 10.“
- Výstupní kanál (Text / Text-to-Speech):** Textová odpověď je primárně zobrazena uživateli na rozhraní. Pokud byl původní vstup hlasový, je textová odpověď následně převedena do zvukové podoby pomocí služby Text-to-Speech a přehrána uživateli, čímž se uzavírá konverzační smyčka.

Celý tento proces elegantně skrývá technickou složitost a poskytuje uživateli iluzi plynulé konverzace s datovým systémem.

Role MCP serverů a Gemini CLI

Klíčovou komponentou pro bezpečnou a spravovatelnou komunikaci mezi LLM a externími systémy (databáze, soubory, API) jsou tzv. **MCP (Model Context Protocol) servery** [16]. Tyto servery fungují jako řízená brána neboli „toolbox“ pro umělou inteligenci. Místo toho, aby měl LLM přímý a nekontrolovaný přístup k datovým zdrojům, MCP server mu poskytuje sadu jasně definovaných *nástrojů* (tools). Každý nástroj reprezentuje konkrétní operaci, například „spust SQL dotaz v databázi ClickHouse“ nebo „přečti soubor z Google Drive“.



Obrázek 1.10.: Model context protocol. Zdroj: modelcontextprotocol.io

Pro interakci s těmito servery a definici dostupného kontextu slouží různé **klientské aplikace** (*MCP Clients/Hosts*). Může se jednat o nástroje příkazové řádky (např. *Gemini CLI*), desktopové aplikace (např. *Claude Desktop App*), vývojová prostředí (IDE jako VS Code, Cursor) nebo webová rozhraní. Tyto aplikace umožňují vývojáři či uživateli „uzemnit“ model tím, že mu zprostředkují připojení ke konkrétním MCP serverům. LLM se následně nedotazuje přímo cílové databáze, ale volá nástroje zpřístupněné skrze klienta a MCP server, který se stará o samotnou exekuci, autentizaci a bezpečnostní logování.

2. Teoretická část

Tato kapitola se zaměřuje na teoretické vymezení klíčových pojmů a principů, které tvoří základ pro praktickou část práce. Cílem je popsat architekturu moderních datových platform, jejich vrstvy a související technologie využívané pro ukládání, zpracování a analýzu dat. Pozornost je věnována především konceptům *datového jezera*, *datového skladu* a technologiím *OLAP*, které tvoří jádro současných analytických systémů. Součástí kapitoly je také přehled nástrojů Business Intelligence a teoretické vymezení *sémantické vrstvy*, jež propojuje datový sklad s prezentační částí platformy a umožňuje jednotnou interpretaci dat napříč organizací.

2.1. Úvod do problematiky datových skladů a OLAP

Návrh systému, který byl v této práci vytvořen, nepředstavuje univerzální ani definitivní řešení. Každý datový architekt by měl vždy zohlednit konkrétní potřeby a podmínky dané organizace. Například některé společnosti nemusí vyžadovat implementaci *sémantické vrstvy* – mohou disponovat vlastním týmem databázových specialistů, kteří vytvářejí databázové pohledy, a datových analytiků, kteří z těchto pohledů následně generují reporty. Jiné organizace naopak tento systém nemohou použít vůbec, například v případě požadavku na zpracování dat v reálném čase, pro které jsou vhodnější jiné technologie a architektury.

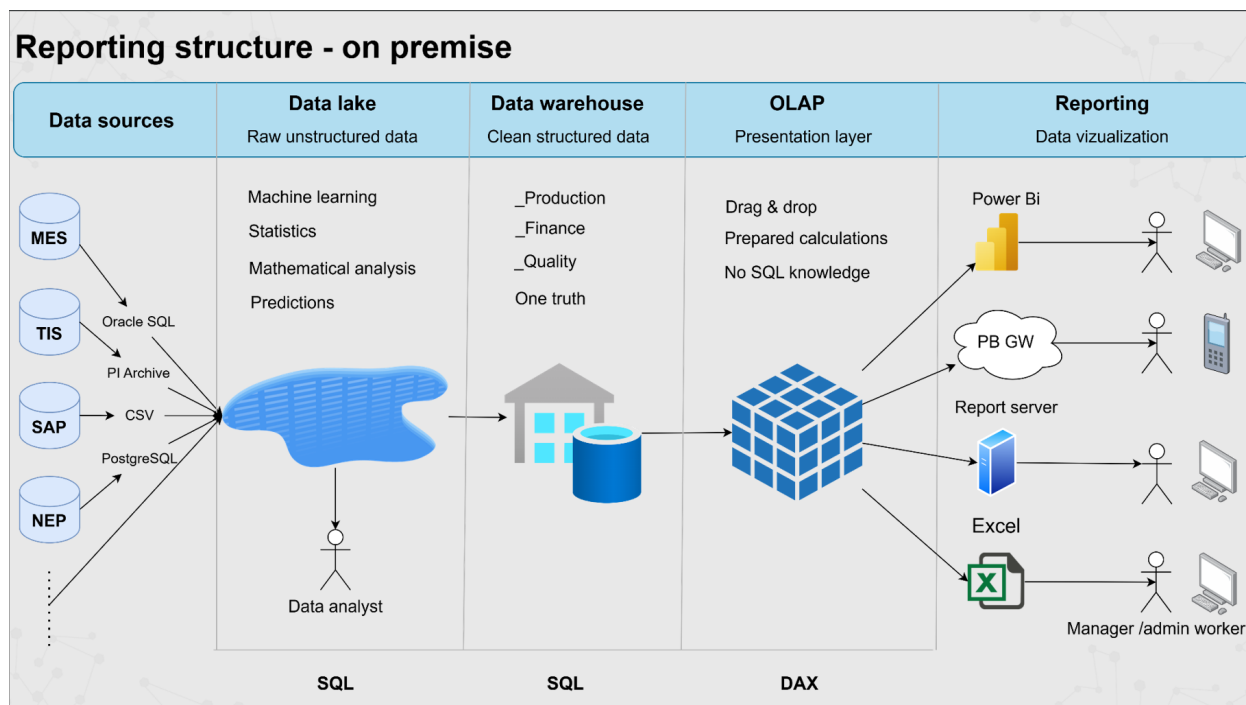
Navržený systém proto představuje jedno z možných řešení, přizpůsobené konkrétní situaci a prostředí, ve kterém byl vyvíjen.

Z hlediska infrastruktury je systém koncipován jako **on-premise** řešení, tedy provozované výhradně na vlastním hardwaru organizace. Většina architektury je realizována pomocí kontejnerizace (Docker), což umožňuje modulární skládání jednotlivých komponent, snadnou správu a replikovatelnost prostředí. Celý systém je rozdělen do pěti vrstev, jak je znázorněno na obrázku 2.1.

Datové zdroje

Základní vstupní vrstvu systému tvoří datové zdroje, které obvykle pocházejí z různých podnikových systémů, jako jsou například MES systémy, CRM nebo ERP. Datový zdroj je zpravidla reprezentován databází, avšak v praxi se často setkáváme také s textovými soubory (například ve formátu CSV). Ačkoliv formát CSV nemusí na první pohled působit jako spolehlivý datový zdroj,

v praxi se používá poměrně často. Důvodem bývá například licenční politika – přímý přístup do databáze může být zpoplatněn, zatímco export dat do CSV souboru bývá dostupný zdarma nebo bez omezení.



Obrázek 2.1.: Navržená struktura systému. Datové zdroje jsou ilustrační. Zdroj: vlastní zpracování autora, dle architektury Mondī Štětí a.s.

Datové jezero

Datové jezero (*Data Lake*) představuje logický koncept centralizovaného úložiště určeného pro uchovávání obrovského množství *surových dat* (*raw data*) v jejich nativním formátu (např. JSON, XML, binární soubory).

Ačkoliv je Datové jezero často asociováno s cloudovými službami (např. AWS S3, Azure Data Lake Storage), tento koncept lze efektivně implementovat i v lokálním (on-premise) prostředí.

V rámci řešeného projektu je Datové jezero implementováno na relační databázi. Přestože relační databáze vyžaduje definici schématu tabulky, je princip *schema-on-read* zachován. To je realizováno tak, že veškerá variabilní data jsou uložena v jediném sloupci (např. *payload*), který je definován jako řetězcový (textový) typ (TEXT nebo VARCHAR). Tím se fyzicky vytvoří pevné schéma pro tabulku, avšak *logický datový typ jednotlivých vnitřních prvků* dat (např. JSON) se určuje staticky / dynamicky až v rámci transformačního (ETL) procesu při jejich parsování a vkládání do Datového skladu. Tato volba zachovává maximální flexibilitu, ačkoliv pokročilejší implementace by mohla využít nativní datové typy pro nestrukturovaná data, jako je JSONB v PostgreSQL.

Klíčové role a cíle Datového jezera:

1. *Konsolidace a Vstupní Zóna (Landing Zone)*: Primární funkcí je konsolidovat data z mnoha heterogenních zdrojů na jedinou platformu, sloužící jako *vstupní zóna* pro surová data před jejich dalším zpracováním.
2. *Oddělení Integrační Logiky*: Umožňuje *oddělit logiku připojení a sběru dat* od vlastního Datového skladu. Tím se zajišťuje, že náročné transformační procesy (ETL/ELT) v Datovém skladu nejsou bezprostředně zatíženy problémy s konektivitou, dostupností zdrojů nebo dynamickou strukturou dat.
3. *Audit a Rodokmen Dat (Data Lineage)*: Uchovávání dat v jejich *původním (surovém) formátu* umožňuje kdykoliv ověřit, z jakých zdrojových dat byla odvozena data v Datovém skladu. To je nezbytné pro *auditní účely* a pro *reprodukcí analytických výsledků* při změnách transformačních pravidel.

Datový sklad

Datový sklad (Data Warehouse, DWH) je centrální, časově závislé úložiště historických i aktuálních dat. Jeho primárním účelem je podpora rozhodovacích procesů. Na rozdíl od provozních databází (OLTP) je struktura DWH optimalizována pro rychlé čtení, agregace a dotazování na velkých objemech dat.

Datové tržiště (Data Mart) je v korporátním prostředí definováno jako *podložka datového skladu* zaměřená na data a metriky potřebné pro specifickou obchodní oblast (např. výroba, kvalita, prodej). Jedná se o menší, tématicky specializovanou entitu, která usnadňuje reportování a analýzu pro konkrétní skupinu uživatelů.

Konceptuální návrh DWH se opírá o dvě hlavní, avšak protichůdné, metodiky:

1. *Metodika Billa Inmona (Top-Down Approach)*: Inmonova metodika je označována jako **Top-Down (shora dolů)**, protože začíná návrhem centrálního, podnikového datového skladu (*Enterprise Data Warehouse, EDW*) [17].
 - **Schema**: EDW je modelováno ve vysoce *normalizované* formě (typicky 3. normální forma, 3NF), což zajišťuje nízkou datovou redundanci a maximální integritu.
 - **Tok dat a tržiště**: Data jsou nejprve ETL procesem načtena do detailního, normalizovaného EDW (centrální zdroj pravdy). *Datové tržiště* se vytváří až *sekundárně* z dat v EDW a jsou denormalizovaná, aby sloužila pro rychlé reportování.
2. *Metodika Ralpha Kimballa (Bottom-Up Approach)*: Kimballova metodika je označována jako **Bottom-Up (zdola nahoru)**, protože se zaměřuje na rychlé dodání řešení pro specifické obchodní procesy [18].
 - **Schema**: Využívá *dimenzionální modelování* (schéma *Hvězda* nebo *Sněhová vločka*), které je záměrně *denormalizované*. To zjednodušuje dotazování a maximalizuje výkon pro OLAP úlohy.

- **Tok dat a tržiště:** Data jsou transformována a ukládána *přímo* do dimenzionálních modelů, které *představují Datová tržiště*. Podnikový datový sklad je pak *logickou unií* (sjednocením) těchto jednotlivých tržišť.

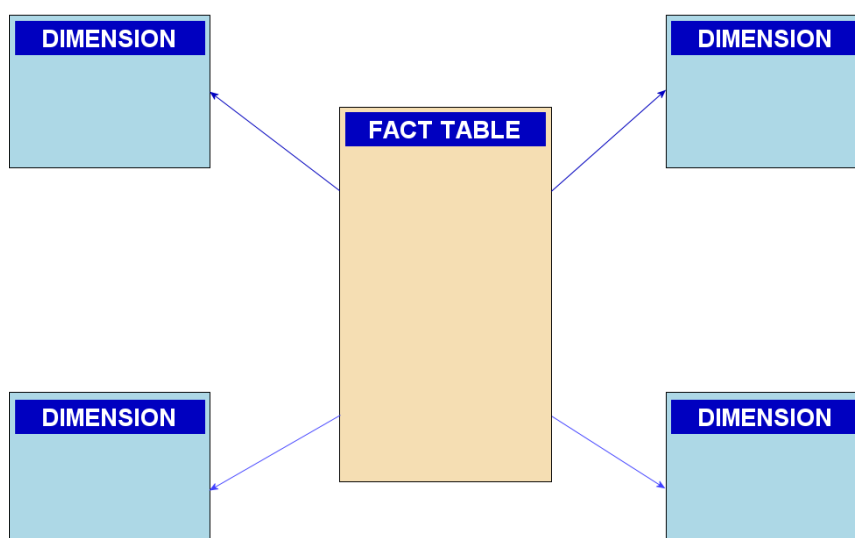
V praxi se však často objevují hybridní systémy kombinující prvky obou přístupů. Takovýto hybridní přístup byl zvolen i v naší implementaci.

Data warehouse z pravidla obsahuje:

- **Staging Area:** Slouží jako dočasné úložiště, kam jsou data přenesena pomocí ETL (Extract, transform and load). V této vrstvě se provádí *harmonizace a validace dat* před aplikací dimenzionálního modelu. Příkladem je tabulka *Stg.CameraCamea* v naší implementaci.
- **Dimenze a Fakta:** Hlavní vrstva organizovaná dle Kimballova modelu (tedy Datový Mart pro tuto analytickou doménu). Skládá se z:
 1. **Dimenze (*Dimensions*):** Uchovávají kontext, atributy a popisné detaily (např. *DimCity*, *DimSensor*).
 2. **Fakta (*Facts*):** Uchovávají měřitelné hodnoty a cizí klíče k dimenzím (např. *FactCameraDetection*).

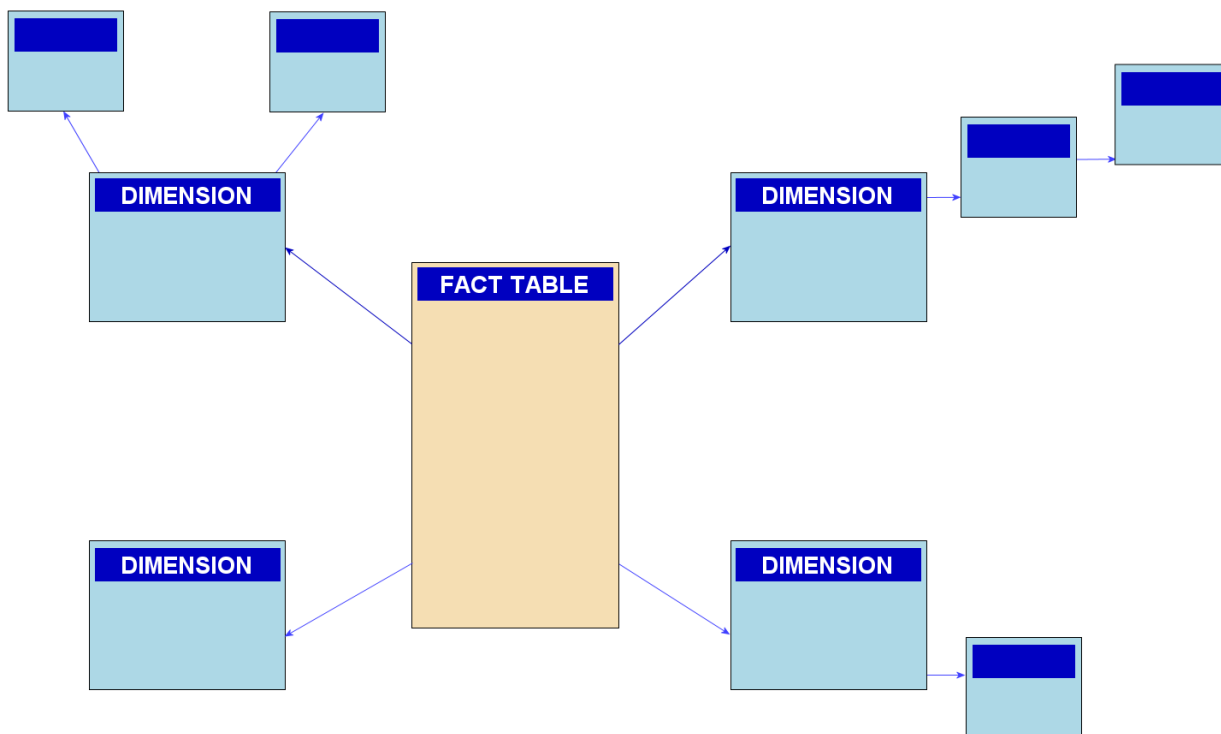
Základními modely používanými pro návrh datového skladu v rámci dimenzionálního modelování (Kimball) jsou:

- **Schéma Hvězda (*Star Schema*):** Jedná se o nejjednodušší a nejčastěji používaný dimenzionální model. Skládá se z centrální tabulky **Faktů** (*Fact Table*), která je obklopena několika tabulkami **Dimenzí** (*Dimension Tables*). Všechny dimenze jsou přímo napojeny na tabulku faktů, čímž vzniká struktura připomínající hvězdu. Vysoká redundance je vyvážena vysokou rychlostí dotazování.



Obrázek 2.2.: Ilustrační schéma hvězdy faktové tabulky a dimenzí. Zdroj: en.wikipedia.org [19]

- **Schéma Sněhová vločka (*Snowflake Schema*):** Jedná se o rozšíření schématu Hvězda, kde některé dimenze jsou *normalizovány* do několika souvisejících tabulek. Tím se snižuje redundance dat, ale na úkor zvýšení složitosti dotazování (je potřeba více JOIN operací), což může mírně zhoršit výkon.



Obrázek 2.3.: Ilustrační schéma sněhové vločky faktové tabulky a dimenzí. Zdroj: en.wikipedia.org [20]

OLAP technologie

OLAP (*Online Analytical Processing*) představuje přístup ke zpracování dat, který umožňuje provádět rychlé analytické dotazy nad velkými objemy informací. Základní myšlenkou OLAP je možnost pohledu na data z více dimenzí – typicky podle času, lokality, zařízení, typu senzoru nebo jiných analytických kritérií. Tento princip umožňuje uživatelům provádět tzv. *slice and dice* operace, tj. analyzovat data z různých perspektiv, agregovat je nebo filtrovat v reálném čase.

Rozlišují se tři tradiční typy OLAP řešení:

- **MOLAP (Multidimensional OLAP)** – pracuje s vlastní vícerozměrnou strukturou dat uloženou mimo relační databázi. Tento přístup poskytuje velmi rychlé odezvy při výpočtech a agregacích díky předpočítaným datovým strukturám (tzv. *cubes*), avšak vyžaduje rozsáhlejší přípravu a větší prostorové nároky.
- **ROLAP (Relational OLAP)** – využívá klasickou relační databázi, nad kterou jsou definovány pohledy a agregační dotazy. Tento přístup je flexibilnější, lépe škálovatelný a umožňuje přímou práci s aktuálními daty bez nutnosti jejich předpočítávání.

- **Tabulární modely (In-Memory)** – moderní přístup, který kombinuje výhody obou předchozích. Data jsou uložena ve vysoce komprimovaném sloupcovém formátu přímo v operační paměti (RAM), což zajišťuje extrémní rychlost odezvy srovnatelnou s MOLAP, přičemž se pracuje s relačním modelem dat (tabulky a vztahy) jako u ROLAP. Typickým zástupcem je Microsoft SSAS Tabular (v režimu Import) nebo Power BI.
- **HOLAP (Hybrid OLAP)** – kombinuje výhody MOLAP a ROLAP, tedy rychlé předpočítané agregace a flexibilitu dotazování nad detailními daty. Tento přístup je často využíván v moderních BI řešeních, kde se pro nejčastěji používané metriky udržují souhrnné cache.

Mezi nejpoužívanější nástroje pro implementaci OLAP vrstev patří například Microsoft SQL Server Analysis Services (SSAS Tabular) jako zástupce komerčního řešení, a z open-source prostředí pak zejména Apache Kylin, Mondrian (součást Pentaho) nebo Cube (dříve Cube.js), který poskytuje moderní API pro analytické dotazy. Některé databázové systémy, jako například ClickHouse, navíc integrují funkce OLAP přímo do svého jádra, což umožňuje provádět agregace v reálném čase bez nutnosti vytvářet samostatnou analytickou vrstvu.

V současnosti je trendem přechod od klasických multidimenzionálních modelů k tabulárním řešením, která nabízejí vyšší flexibilitu, lepší integraci s nástroji pro vizualizaci dat a nižší nároky na údržbu datového modelu.

Sémantická vrstva

Sémantická vrstva (*Semantic Layer*) představuje logickou nadstavbu nad datovým skladem, jejímž hlavním cílem je sjednotit přístup k datům napříč celou organizací. Slouží jako prostředník mezi technickou strukturou databáze a uživatelskými nástroji Business Intelligence. Namísto přímé práce s databázovými tabulkami umožňuje uživatelům přistupovat k datům prostřednictvím předdefinovaných metrik, dimenzí a vztahů, které odpovídají obchodní logice organizace.

Tímto způsobem sémantická vrstva abstrahuje technické detaily a umožňuje vytváření analytických dotazů bez nutnosti znalosti SQL či fyzického modelu dat. V praxi tento koncept implementují moderní frameworky jako *Cube.js*, *dbt Semantic Layer*¹ nebo *Looker Semantic Model*², které poskytují rozhraní pro standardizovaný přístup k datům prostřednictvím API nebo SQL proxy [3].

K hlavním přínosům sémantické vrstvy patří:

- **Konzistence metrik a výpočtů:** Veškeré reporty a dashboardy využívají jednotně definované ukazatele, čímž se eliminuje riziko rozdílné interpretace dat.
- **Zjednodušení analytické práce:** Uživatelé mohou tvořit vizualizace nebo reporty bez nutnosti znalosti komplexní struktury datového skladu.
- **Zvýšení výkonu a bezpečnosti:** Sémantická vrstva často zajišťuje cachování a řízení přístupů na úrovni obchodních objektů, což zlepšuje odezvu systému a kontrolu nad daty.

¹<https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl>

²<https://cloud.google.com/looker/docs/semantic-model-overview>

Z pohledu architektury datových platforem tak sémantická vrstva představuje klíčový prvek mezi datovým skladem a vizualizační vrstvou, který propojuje technologickou přesnost s obchodní srozumitelností.

2.2. Nástroje Business Intelligence

Nástroje Business Intelligence (BI) představují softwarové řešení určené k transformaci surových dat na informace, které podporují rozhodovací procesy na všech úrovních řízení. Cílem BI systémů je zajistit uživatelům přístup k aktuálním, konzistentním a relevantním datům, a to formou interaktivních vizualizací, reportů či analytických přehledů. V rámci moderních datových platforem tvoří BI nadstavbu nad datovým skladem a OLAP vrstvou. Zprostředkovává uživatelům přístup k datům v srozumitelné a vizuálně přitažlivé podobě a umožňuje jejich interpretaci bez nutnosti znalosti technických detailů implementace.

BI nástroje dnes představují klíčový prvek datově řízeného rozhodování a zahrnují široké spektrum funkcí – od základních přehledů a dashboardů až po pokročilé analytické a prediktivní modely využívající strojové učení.

Funkční oblasti nástrojů Business Intelligence

Nástroje BI lze z hlediska jejich zaměření rozdělit do několika funkčních oblastí, které společně pokrývají celý proces přeměny dat na znalosti:

- **Reportování a vizualizace dat** – umožňuje prezentaci dat formou tabulek, grafů a interaktivních dashboardů. Uživatelé tak mohou rychle identifikovat klíčové ukazatele výkonnosti (KPI), sledovat trendy a vyhodnocovat vývoj v čase.
- **Ad-hoc analýza a průzkum dat** – nabízí uživatelům možnost vytvářet vlastní pohledy na data, provádět filtrování, seskupování nebo detailní analýzy (tzv. *drill-down* a *drill-up*). Tato oblast je důležitá pro datové analytiky a business specialisty, kteří potřebují flexibilně reagovat na nové otázky bez zásahu IT oddělení.
- **Pokročilá analytika a prediktivní modelování** – integruje BI s nástroji pro datovou vědu a strojové učení. Umožňuje například předpověď trendů, klasifikaci nebo detekci anomálií. Tyto funkce bývají častější u komerčních řešení, která využívají propojení s cloudovými službami nebo integrované modelovací prostředí (např. Microsoft Azure Machine Learning).
- **Sdílení a spolupráce** – moderní BI nástroje podporují publikaci reportů a dashboardů napříč organizací, správu oprávnění, exporty a integraci s komunikačními nástroji (např. Microsoft Teams, Slack, Power BI report server, Apache Superset web).

Přehled dostupných BI nástrojů

Z pohledu dostupných řešení lze nástroje Business Intelligence rozdělit do dvou hlavních kategorií – open-source a komerčních platform. Obě skupiny mají své výhody a nevýhody, které je nutné zohlednit při rozhodování o jejich implementaci v konkrétním prostředí.

- **Open-source řešení** – nástroje jako *Grafana*, *Apache Superset*, *Metabase* nebo *Redash* poskytují široké možnosti integrace s relačními i nestrukturovanými datovými zdroji. Jsou vhodné zejména pro organizace, které preferují flexibilitu, nižší licenční náklady a možnost přizpůsobit systém vlastním potřebám. Nevýhodou bývá nutnost technické správy, složitější počáteční konfigurace a omezená úroveň uživatelské podpory.
- **Komerční řešení** – mezi nejrozšířenější komerční platformy patří *Microsoft Power BI*, *Tableau* a *Qlik Sense*. Tyto nástroje se vyznačují vysokou mírou integrace s podnikovými systémy, širokými možnostmi automatizace a pokročilými funkcemi pro datové modelování, sdílení a prediktivní analýzy. Výhodou bývá profesionální podpora výrobce, pravidelné aktualizace a integrace s cloudovými službami. Na druhé straně představují komerční nástroje vyšší finanční náklady a častou závislost na dodavateli (tzv. *vendor lock-in*).

Kritéria výběru BI nástroje

Volba vhodného BI nástroje závisí na potřebách organizace, technické infrastruktury a rozpočtu. Při rozhodování je vhodné zohlednit následující kritéria:

1. **Uživatelská přívětivost a intuitivní rozhraní** – nástroj by měl umožnit tvorbu reportů i netechnickým uživatelům bez nutnosti psaní SQL dotazů.
2. **Konektivita k datovým zdrojům** – podpora přímého připojení k datovému skladu, OLAP vrstvám či externím API.
3. **Automatizace a aktualizace dat** – schopnost plánovat obnovu dat, vytvářet datové toky (pipelines) a spravovat přístupová oprávnění.
4. **Možnosti spolupráce a sdílení** – integrace s podnikovými systémy (např. Microsoft Teams, Slack, SharePoint) a možnost publikování interaktivních dashboardů.
5. **Licenční model a celkové náklady na provoz (TCO)** – kromě pořizovací ceny je třeba zohlednit náklady na údržbu, školení uživatelů a případnou infrastrukturu.

V praxi organizace často kombinují více nástrojů – například open-source řešení pro interní analýzy a komerční systém pro prezentaci výsledků managementu. Tento přístup umožňuje optimalizovat náklady a zároveň využít silných stránek jednotlivých platform.

2.3. Přehled a charakteristika využitých technologií

Tato sekce vymezuje technologie, na které se práce dále zaměřuje. Výběr zahrnuje zástupce komerčních i open-source přístupů, jejichž charakteristika a role v kontextu navržené architektury je shrnuta v následující tabulce.

Vrstva: Datový sklad	
Open-source řešení	Komerční řešení
PostgreSQL a TimescaleDB PostgreSQL je open-source relační databázový systém s podporou SQL standardu. Klíčovým aspektem pro účely této práce je jeho rozšíření <i>TimescaleDB</i>, které PostgreSQL transformuje na vysoce výkonnou databázi pro časové řady. Tato kombinace je vhodná pro telemetrická data, jako jsou data ze senzorů a kamer, díky optimalizacím pro časově orientované dotazy a efektivní kompresi dat. ClickHouse ClickHouse je open-source, sloupcově orientovaný databázový systém navržený primárně pro online analytické zpracování (OLAP). Jeho architektura je optimalizována pro extrémně rychlé provádění agregačních dotazů nad velkými objemy dat. V kontextu srovnání představuje zástupce vysoce výkonných analytických databází nové generace, které umožňují analýzu dat v reálném čase bez nutnosti rozsáhlých předagregací. MariaDB MariaDB je open-source relační databázový systém, který vznikl jako fork MySQL a zachovává si s ním vysokou kompatibilitu. Významným prvkem je jeho rozšíření <i>MariaDB ColumnStore</i>, které transformuje tradiční řádkově orientovanou databázi na hybridní systém schopný sloupcového ukládání dat. Tato architektura je optimalizována pro analytické dotazy (OLAP).	Microsoft SQL Server Microsoft SQL Server (MSSQL) je komplexní komerční systém pro správu relačních databází (RDBMS). Kromě standardních transakčních (OLTP) operací nabízí robustní nástroje pro datové sklady, včetně integračních služeb (SSIS) a analytických služeb (SSAS). V kontextu této práce představuje MSSQL referenční komerční řešení, konkrétně v roli databázového engine pro datový sklad a zároveň jako platforma pro SSAS Tabular model.

Vrstva: Sémantická vrstva / OLAP	
Open-source řešení	Komerční řešení
Cube.js Cube.js je open-source sémantická vrstva (semantic layer), jejímž hlavním úkolem je abstrahovat technickou komplexitu datového skladu a poskytovat konzistentní business logiku. Definuje datové modely, metriky (measures) a dimenze pomocí JavaScriptu. Klíčovou vlastností je poskytování standardizovaného rozhraní (API), včetně emulace <i>PostgreSQL SQL API</i>. To umožňuje, aby se vizualizační nástroje (jako Apache Superset) připojovaly ke Cube.js stejným způsobem, jako by se připojovaly k běžné databázi, čímž se výrazně zjednodušuje integrace a zvyšuje výkon díky pokročilému cachování a předagregacím.	Microsoft SQL Server Analysis Services SSAS je komponenta Microsoft SQL Serveru určená pro OLAP a Business Intelligence. Tato práce se zaměřuje na jeho <i>Tabulární model (Tabular Model)</i>, který funguje jako <i>in-memory</i> databáze optimalizovaná pro analytické dotazy pomocí jazyka DAX (Data Analysis Expressions). Slouží jako komerční protějšek k Cube.js, poskytující podobnou sémantickou vrstvu s pokročilými možnostmi bezpečnosti, správy a hlubokou integrací s nástroji Microsoft ekosystému.

Vrstva: Vizualizace / BI	
Open-source řešení	Komerční řešení
Apache Superset Apache Superset je moderní open-source platforma pro vizualizaci dat a business intelligence. Umožňuje snadné připojení k široké škále datových zdrojů (včetně SQL API poskytovaného Cube.js) a intuitivní tvorbu interaktivních dashboardů a reportů. V rámci navržené architektury reprezentuje flexibilní open-source řešení pro vizualizační vrstvu, které klade důraz na samoobslužnou analýzu a customizaci.	Microsoft Power BI Power BI je komerční analytický nástroj od společnosti Microsoft, který představuje průmyslový standard pro vizualizaci dat a business intelligence. Vyniká těsnou integrací s celým ekosystémem Microsoftu, zejména s SSAS Tabular, což umožňuje efektivní analýzu, sdílení reportů a spolupráci v rámci organizace. V kontextu této práce slouží jako reprezentant komerční vizualizační platformy s pokročilými funkcemi pro datovou transformaci, modelování a distribuci obsahu.

3. Praktická část

Praktická část této práce se zaměřuje na návrh, implementaci a ověření metodiky pro srovnání open-source a komerčních nástrojů využitelných pro analýzu a vizualizaci dat na datové platformě Portabo. Cílem bylo vytvořit plně funkční datový sklad s podporou OLAP analýz, který umožňuje zpracovávat reálná data poskytovaná datovým centrem Ústeckého kraje a připravit prostředí pro následné výkonové a funkční porovnání vizualizačních nástrojů.

V rámci řešení byly navrženy a implementovány datové toky, které respektují jednotnou architekturu systému složeného z vrstev *datového jezera*, *datového skladu* (staging vrstva, dimenze, fakta), *OLAP vrstvy* a *reportingu*. Tato struktura byla zachována napříč všemi testovanými technologiemi, přičemž pro jednotlivé implementace byl použit stejný princip ETL zpracování, stejný způsob transformace a obdobné rozvržení datového modelu založeného na dimenzionálním schématu.

Data využitá pro testování pocházejí převážně z datového toku města Bílina, konkrétně z MQTT témat obsahujících telemetrické údaje o kamerových detekcích vozidel. V případě implementace nad PostgreSQL/TimescaleDB byla navíc použita širší množina dat zahrnující také Wi-Fi senzory, elektrické měřiče a další zdroje. Tyto zprávy byly zpracovány pomocí implementovaných ETL procesů a uloženy do datového skladu vytvořeného nad několika databázovými technologiemi (PostgreSQL/TimescaleDB, MariaDB, Microsoft SQL Server a ClickHouse). Každá z těchto technologií představuje odlišný přístup – od komerčního řešení po výkonné open-source systémy určené pro analytické zpracování dat.

Příprava a migrace datové základny

Výchozím bodem pro naplnění datového jezera (Data Lake), které sloužilo jako zdroj pro všechny následné ETL procesy, byl poskytnutý SQL dump určený pro databázi MariaDB. Jelikož metodika srovnání zahrnovala i jiné databázové systémy, bylo nutné provést migraci těchto dat do jednotlivých cílových prostředí. Pro každou technologii byl zvolen specifický přístup:

Microsoft SQL Server: Pro migraci do MSSQL byl zvolen dvoukrokový přístup, neboť původní pokusy o vytvoření vlastního konverzního skriptu se ukázaly jako nespolehlivé.

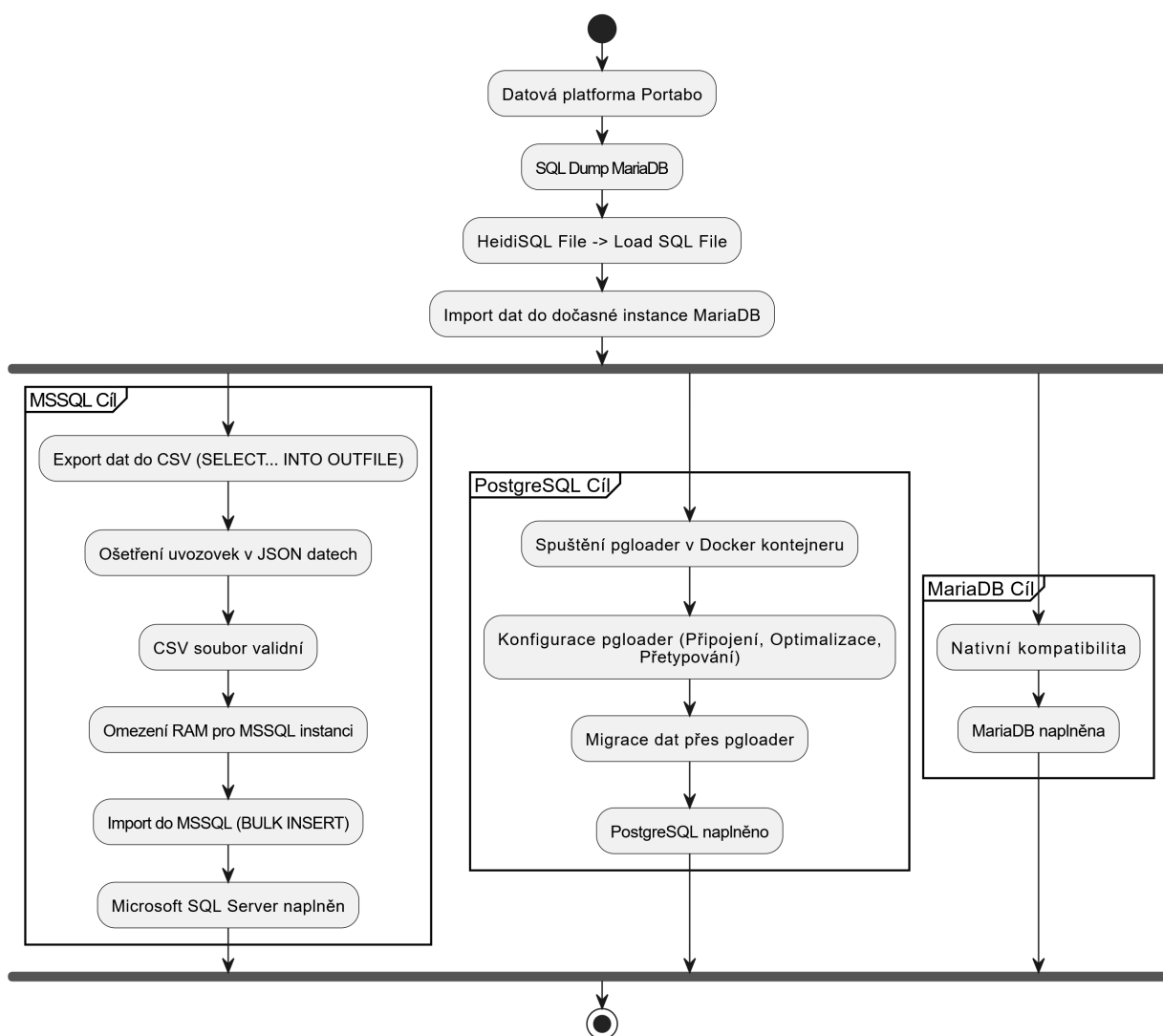
1. **Export z MariaDB:** Data byla nejprve naimportována do dočasné instance MariaDB. Následně byla z této databáze exportována do univerzálního formátu CSV pomocí SQL dotazu `SELECT . . . INTO OUTFILE`. Tento export vyžadoval specifické ošetření uvozovek v JSON datech, aby byl výsledný soubor validní.

2. **Import do MSSQL:** Výsledný CSV soubor byl naimportován do MSSQL pomocí příkazu `BULK INSERT`. Během tohoto procesu bylo také nutné manuálně omezit maximální přidělenou operační paměť (RAM) pro instanci MSSQL serveru, aby nedošlo k zahlcení systémových prostředků.

PostgreSQL: Pro migraci do PostgreSQL byl využit nástroj `pgloader` [21], který umožňuje přímou migraci mezi různými databázovými systémy.

- **Konfigurace a spuštění:** Nástroj byl spuštěn v rámci vlastního Docker kontejneru. Konfigurační soubor definoval přímé připojení ke zdrojové MariaDB databázi (`host.docker.internal:3306`) a cílové PostgreSQL databázi (`host.docker.internal:5433`).
- **Optimalizace výkonu:** Konfigurace `pgloader` byla optimalizována pro co nejvyšší rychlost přenosu. Bylo využito paralelní zpracování (`workers = 4`, `concurrency = 4`), dávkování dat po 50 000 řádcích (`batch rows = 50000`) a navýšení operační paměti pro proces migrace (`maintenance_work_mem` to `'512MB'`).
- **Přetypování dat:** Součástí konfiguračního souboru byla také explicitní pravidla pro přetypování datových typů mezi oběma systémy, zejména pro správnou interpretaci časových údajů (`CAST type datetime to timestamptz...`).

Těmito postupy bylo zajištěno, že identická datová základna (datové jezero) byla k dispozici ve všech testovaných databázových systémech.



Obrázek 3.1.: Aktivita diagram nahrání MariaDB dumpu do odlišných databází. Zdroj: vlastní zpracování pomocí aplikace PlantUML Editor [13]

3.1. Návrh metodiky porovnání

Cílem metodiky porovnání je zajistit objektivní a opakovatelné vyhodnocení open-source a komerčních technologií využitelných pro analýzu a vizualizaci dat v prostředí platformy Portabo. Porovnání je založeno na principech jednotného datového toku, jednotného datového modelu a shodného způsobu zpracování dat. Tím je zaručeno, že rozdíly ve výsledcích budou dány samotnými vlastnostmi testovaných technologií, nikoli odlišnostmi v implementaci.

Každá z porovnávaných variant bude hodnocena z následujících hledisek:

- **Výkonové parametry** – rychlost odezvy analytických dotazů, vliv indexace a pre-agregací na výkon, srovnání architektur MOLAP a ROLAP;
- **Funkční vlastnosti** – možnosti vizualizace včetně práce s geoprostorovými daty, architektura reportingu a interaktivita;
- **Uživatelská přívětivost** – cílová skupina uživatelů, náročnost implementace a flexibilita nástrojů;
- **Ekonomické aspekty** – celkové náklady na vlastnictví (TCO) zahrnující licence, infrastrukturu a lidský kapitál.

Metodika tedy určuje rámec, v němž budou jednotlivé technologie nasazeny, naplněny shodnými daty a následně testovány.

3.2. Příprava dat a návrh datového skladu

Analýza a příprava zdrojových dat

V úvodní fázi implementace byla vybrána konkrétní datová doména, nad kterou bude systém postaven. V tomto případě se jedná o kamerový systém města Bílina. Cílem bylo ověřit, zda je možné tato data zpracovávat a následně připravit půdu pro jejich dynamické zpracování.

Poskytnutý MariaDB SQL dump obsahoval následující sloupce: `id(bigint(11))`, `time(timestamp)`, `topic(tinytext)` a `payload(text)`. Tyto údaje představovaly surová telemetrická data, původně pocházející z MQTT komunikace, která byla exportována do databázového dumpu. Tento dump sloužil jako vstupní datová sada pro analýzu a testování datového toku.

id	time	topic	payload
127.537.462	2024-06-26 04:00:01.000 +0200	/mve/MVElenesice/TG1:_l_protok	["compound_alias":"TG1:_l_protok","compound_name":"TG1:_l_protok","device_id":"2","display_value":"9.6","limits":{"defined":false},"defined":false},"defined":false},"defined":false]"]
127.537.464	2024-06-26 04:00:03.000 +0200	/vodomyedecin	["misto":"msmoskevka","stav":"1171663"]
127.537.465	2024-06-26 04:00:04.000 +0200	/senzory/wifi/co2/12	["end_device_ids":{"device_id":"wifi-co2-12"},"received_at":"2024-06-26T00:00:04Z","uplink_message":{"decoded_payload":{"sensor":"wifi-co2"},"msg":"1246128.7639.23","format":"text"}]
127.537.466	2024-06-26 04:00:04.000 +0200	/tsndata/senzory/wifi/co2/12	["end_device_ids":{"device_id":"wifi-co2-12"},"received_at":"2024-06-26T00:00:04Z","uplink_message":{"decoded_payload":{"sensor":"wifi-co2"},"msg":"1246128.7639.23","format":"text"}]
127.537.468	2024-06-26 04:00:06.000 +0200	/tsndata/eui-24e124785e095772-zzs-018	["end_device_ids":{"device_id":"eui-24e124785e095772-zzs-018"},"application_ids":{"application_id":"zachranka-lora"},"dev_eui":"24E124785E095772","join_eui":"70B3D57ED0606D60"]
127.537.470	2024-06-26 04:00:07.000 +0200	/vodomyedecin	["misto":"zsbzeovasklep","stav":"1216886"},"misto":"zsbzeovastny","stav":"1038661"]]
127.537.471	2024-06-26 04:00:08.000 +0200	/vodomyedecin	["misto":"zskolnici","stav":"124871174"]]
127.537.472	2024-06-26 04:00:09.000 +0200	/homie/nepro1:3/\$state	["alert"]

Obrázek 3.2.: Struktura tabulky `mqttentries` obsahující zdrojová telemetrická data. Zdroj: vlastní s použitím aplikace DBeaver [22]

Z obrázku je patrná struktura vstupních dat. Ze sloupce `topic` lze částečně odvodit, o jaký typ senzoru se jedná. Podrobnější analýza však ukázala, že názvy témat nejsou konzistentní. Například kamera z Bíliny může mít téma `/Bilina/kamery/comea/BI-TP-02`, zatímco vodoměr je označen jednoduše `/vodomery/decin`. V databázi se rovněž vyskytují senzory, které vykonávají stejnou funkci, avšak používají odlišné pojmenování.

Z těchto důvodů bylo vyhodnoceno, že vytvoření skriptu, který by dynamicky parsoval JSON struktury na základě názvů témat, není vzhledem k nekonzistenci dat reálně možné. Bylo nutné zvolit robustnější přístup založený na analýze obsahu samotných zpráv.

Automatizovaná analýza JSON struktur

Pro systematickou analýzu JSON [23] struktur byl vytvořen vlastní skript v jazyce Python, `analyze_json.py`. Tento skript seskupuje MQTT témata do logických celků na základě strukturální podobnosti jejich JSON schémat ve sloupci `payload`.

K porovnání struktury dvou JSON objektů byla využita **Jaccardova podobnost** [24], která měří poměr velikosti průniku a sjednocení množin jejich klíčů. Analýza s prahovou hodnotou 100 % shody odhalila větší množství identických struktur, avšak některé senzory se lišily pouze v několika atributech – pravděpodobně kvůli odlišným verzím firmwaru. Proto byla hodnota podobnosti postupně snižována až na 50 %, čímž se podařilo identifikovat širší spektrum vzájemně příbuzných JSON struktur.

Výstupem této první fáze analýzy jsou skupiny témat, které sdílejí podobnou datovou strukturu, jak je znázorněno na obrázku 3.3.

GroupID	NumTopic	Topics	Keys
7	1254	/Energo/DCUK/INEPRO-Pro380-Mod/inepro1-2/ApparentPower, /Energo/DCUK/INEPRO-Pro380-Mod/inepro1-2/Appa	value:value
22	288	/tntdata/eui-24e124136c489089-02, /tntdata/eui-24e124136c489130-03, /tntdata/eui-24e124136c489175-01, /tntdata	correlation_ids:object_array:value, end_device_ids:object_application_ids:object_application
26	169	/udp1881/15102001, /udp1881/15102002, /udp1881/15102004, /udp1881/15102005, /udp1881/15102006, /udp1881/1	msgid:object:value, session:object_id:object:value, session:object_type:object:value, batter
1	31	/Bilina/kamery/comea/BI-MO-11, /Bilina/kamery/comea/BI-MO-11B, /Bilina/kamery/comea/BI-MO-01, /Bilina/kamery	detectiontype:object:value, ilpc:object:value, lp:object:value, sensor:object:value, utc:object:va
25	14	/tntdata/voda/eui-24e124713d167798, /tntdata/voda/eui-24e124713d321242-vzdalenost-06, /tntdata/voda/eui-24e1	app_id:object:value, battery:object:value, dev_id:object:value, distance:object:value, time:obje
21	12	/senzory/wifi/co2/11, /senzory/wifi/co2/12, /senzory/wifi/co2/13, /senzory/wifi/co2/14, /senzory/wifi/co2/69, /senzo	end_device_ids:object_device_id:object:value, received_at:object:value, uplink_message:objec
17	3	/mve/MVELibochovice, /mve/libochovice, /mve/patek	gen1vykcin:object_array:value, gen2vykcin:object_array:value, hlajezmm:object_array:value, f
18	3	/mve/MVELenesice/NS-DI_hl_horni, /mve/MVELenesice/TG1-I_prutok, /mve/MVELenesice/TG2-I_prutok	alias:object:value, archived:object:value, compound_alias:object:value, compound_name:object
5	2	/detektory/video/brna/brnaIF, /detektory/video/brna/cyklotrasa	format:object:value, msg:object_countdata:object_data:object_array:scitani:object_left_> rig

Obrázek 3.3.: Prvotní seskupení témat na základě 50 % strukturální podobnosti JSON schémat. Zdroj: vlastní s použitím aplikace Excel

V dalším kroku bylo nutné tyto automaticky vygenerované skupiny ručně zrevidovat. Ukázalo se, že některé skupiny, ačkoliv měly mírně odlišnou JSON strukturu, patřily ke stejnému typu zařízení, které pouze měřilo jinou veličinu (např. jeden senzor měřící napětí a druhý proud). Tyto logicky související skupiny byly následně sloučeny do finálních celků určených pro ETL zpracování. Tento proces je znázorněn na obrázku 3.4.

GroupID	NumTopic	Topics	Keys
8	2	/Energo/DCUK/SML133/SML133-01, /Energo/DCUK/SML133/SML133-01/act	subjson:object__3:object:value, subjson:object__cos1:object:value, subjson:object__cos2:obje
10	2	/Energo/DCUK/SML133/SML133-01/har, /Energo/DCUK/SML133/SML133-01/osc	subjson:object_delta:object:value, subjson:object_device:object:value, subjson:object__f1:0

Obrázek 3.4.: Tyto logické skupiny byly navíc ještě manuálně sloučeny napříč skupiny. Zdroj: vlastní s použitím aplikace Excel

Návrh a principy ETL procesů

Na základě analýzy dat byly navrženy a implementovány samostatné ETL skripty pro jednotlivé databázové technologie. Každý z těchto skriptů zajišťuje přenos dat z *datového jezera* (surová MQTT data) do *staging vrstvy datového skladu*, kde jsou data očištěna a standardizována. Následně jsou zpracovaná data přesunuta do faktové tabulky `FactCameraDetection`, která tvoří jádro analytické vrstvy systému.

Všechny implementace sdílejí jednotný princip inkrementálního načítání dat. Každý běh ETL skriptu začíná načtením posledního zpracovaného identifikátoru z tabulky

`ETL_IncrementalControl` (sloupec `LastLoadedID`). Tímto způsobem se při každém dalším běhu zpracovávají pouze nově přijaté zprávy. Průběh každého běhu je zapisován do tabulky `ETL_RunLog`, kde jsou evidovány údaje jako název úlohy, čas spuštění a stav (`RUNNING`, `SUCCESS` či `FAILED`).

Data jsou zpracovávána v dávkách (tzv. *batchích*), jejichž velikost je řízena parametrem `BATCH_SIZE`. Každý skript používá databázové kurzory a příkazy `fetchmany()`, aby se minimalizovalo zatížení paměti. Klíčovým polem v těchto datech je `payload`, obsahující JSON strukturu s detaily detekce. Pro jeho zpracování byly v rámci práce implementovány dva odlišné přístupy:

- **Statické ETL (SQL Server, MariaDB, MariaDB+ClickHouse):** Tento přístup byl implementován pro data z kamerového systému města Bílina (`/Bilina/kamery/comea/%`). Vytvořené skripty využívají předem definovanou znalost struktury JSON dat a atributy z `payload` jsou mapovány na pevně dané sloupce ve staging tabulce.
- **Dynamické ETL (PostgreSQL s TimescaleDB):** Naopak pro implementaci nad databází PostgreSQL byl zvolen univerzální přístup. Skript (`pg_timescale_lake_to_staging.py`) byl navržen tak, aby se dokázal automaticky adaptovat na různorodé JSON struktury. Během zpracování dynamicky analyzuje obsah `payload`, odvozuje datové typy a v případě potřeby sám upravuje cílovou staging tabulku příkazem `ALTER TABLE`.

Při zpracování dat jsou prováděny převody datových formátů, ošetřovány chybějící hodnoty a celý proces je obalen v transakcích a blocích `try/except` pro zajištění konzistence a logování chyb.

Implementace ETL pro jednotlivé technologie

SQL Server ETL

ETL proces pro SQL Server je dvoukrokový a obsluhují jej dva samostatné skripty.

- **Krok 1: Z datového jezera do stagingu**
`mssql_bilina_kamery_lake_to_staging.py` skript využívá knihovnu `pyodbc` [25]. Inkrementálně načítá surová data z tabulky `mqttentries`, parsuje JSON `payload` a očištěné záznamy vkládá po dávkách do staging tabulky `[Stg].[CameraCamea]`. Každý běh je logován v tabulce `ETL_RunLog`, přičemž pro získání ID běhu využívá T-SQL klauzuli `OUTPUT inserted.RunID`.

- **Krok 2: Ze stagingu do dimenzí a faktů**

`mssql_bilina_kamery_staging_to_fact.py` skript zpracovává data připravená ve stagingu. Pro nalezení nových dimenzních hodnot nejprve vloží unikátní záznamy z dávky do dočasné tabulky `##TempDimensions`. Následně pomocí T-SQL klauzule `EXCEPT` porovná obsah dočasné tabulky s existujícími dimenzemi a vloží pouze chybějící záznamy. Poté konstruuje záznamy pro faktovou tabulku `FactCameraDetection` pomocí `LEFT JOIN` na dimenze.

MariaDB + MariaDB ColumnStore ETL

Implementace pro MariaDB se řídí stejnou dvoukrokovou logikou, avšak krok 2 je přizpůsoben pro specifika enginu ColumnStore.

- **Krok 1: Z datového jezera do stagingu**

`maria_bilina_kamery_lake_to_staging.py` skript používá knihovnu `pymysql` [26]. Načítá data z datového jezera, zpracovává JSON a ukládá je do staging tabulky `Stg_CameraCamea`. Pro aktualizaci kontrolní tabulky využívá MySQL klauzuli `ON DUPLICATE KEY UPDATE`.

- **Krok 2: Ze stagingu do dimenzí a faktů**

`maria_bilina_kamery_staging_to_fact.py` skript používá pro plnění dimenzí příkaz `INSERT IGNORE`, který přeskočí již existující záznamy. Při plnění faktové tabulky skript nejprve v Pythonu načte všechny dimenzní klíče do paměti, následně projde staging data, nahradí původní hodnoty číselnými klíči a celý výsledek zapíše do dočasného CSV souboru. Nakonec je tento soubor nahrán do faktové tabulky pomocí příkazu `LOAD DATA LOCAL INFILE`.

V kontextu enginu ColumnStore, který nativně nepodporuje auto-inkrementaci, existují dvě hlavní strategie pro generování primárních klíčů. V této práci byla zvolena cesta *generování klíčů přímo v ETL procesu*. Alternativní možností je využití řádkového enginu InnoDB (který auto-inkrementaci podporuje) pro prvotní nahrání dat, následné odstranění indexů a finální přepnutí tabulky na sloupcový formát (`ALTER TABLE ... ENGINE=ColumnStore`). Nicméně tato strategie nebyla pro můj případ vyhovující.

PostgreSQL + PostgreSQL TimescaleDB ETL

ETL proces pro PostgreSQL je unikátní svým dynamickým přístupem v prvním kroku a hromadným vkládáním ve druhém.

- **Krok 1: Dynamické ETL z jezera do stagingu**

`pg_timescale_lake_to_staging.py` skript automaticky analyzuje JSON payload. Pomocí funkcí `flatten_json()` a `infer_pg_type()` dynamicky odvozuje schéma a datové typy. Pokud narazí na nový atribut, sám upraví cílovou staging tabulku příkazem `ALTER TABLE`.

- **Krok 2: Ze stagingu do dimenzí a faktů**

`pg_timescale_staging_to_fact.py` skript využívá pro plnění dimenzí specifickou vlastnost PostgreSQL – příkaz `INSERT ... ON CONFLICT DO NOTHING`. Tento příkaz je spouštěn pomocí optimalizační funkce `execute_values` z knihovny `psycopg2` [27], která sestaví jediný rozsáhlý `INSERT` příkaz obsahující všechny hodnoty najednou. Toto umožňuje databázi zpracovat celou dávku v jediné operaci. Následně jsou data ze stagingu pomocí standardního `INSERT INTO ... SELECT` s `LEFT JOINy` transformována do faktové tabulky.

MariaDB + ClickHouse ETL

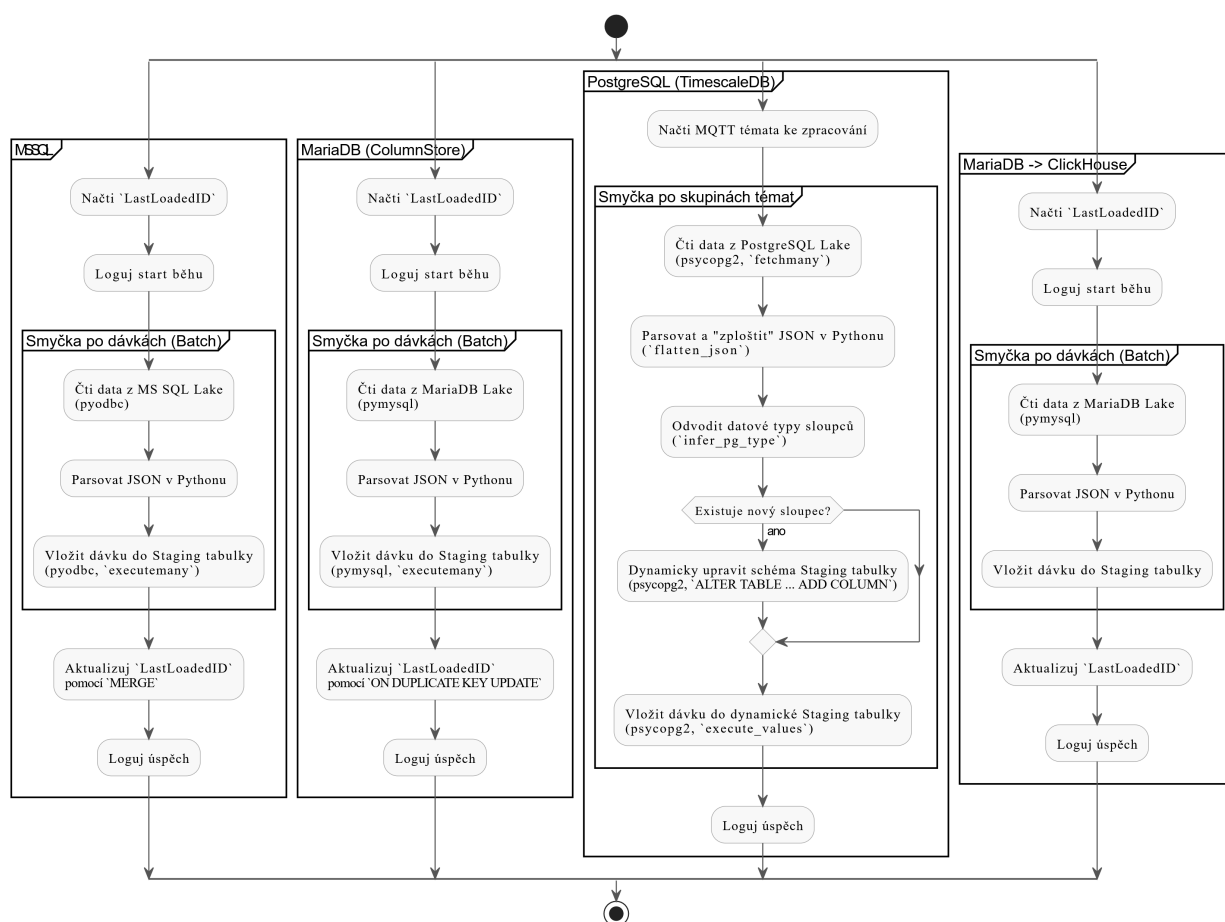
Tato varianta se odlišuje přímým datovým mostem mezi dvěma různými technologiemi a specifickým způsobem přenosu dat.

- **Krok 1: Z datového jezera do stagingu**

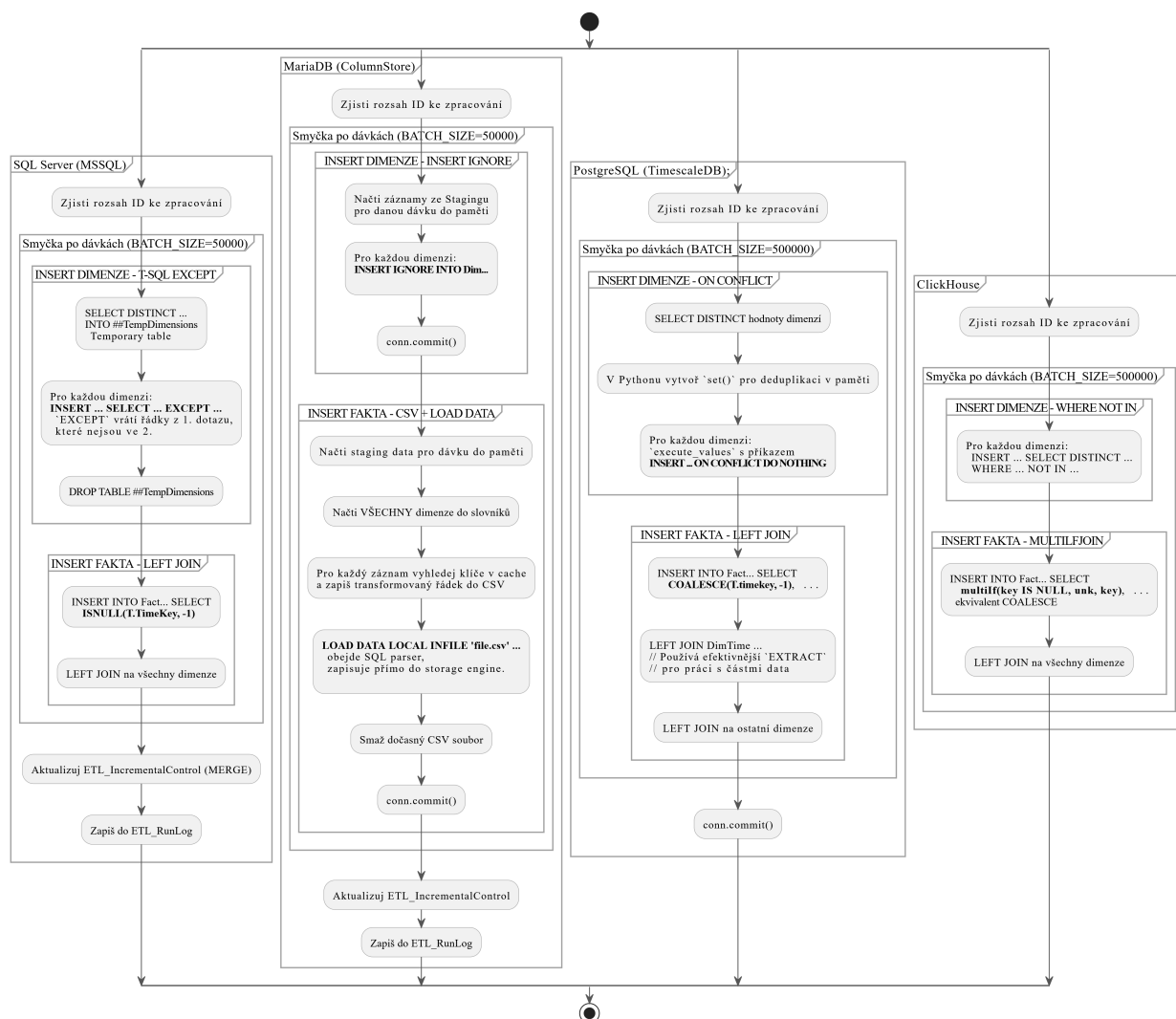
`maria_click_bilina_kamery_lake_to_staging.py` skript zajišťuje přenos dat z MariaDB do ClickHouse. Pro zápis do ClickHouse je využit nativní binární protokol (port 9000), přičemž data jsou v Pythonu agregována do dávky záznamů (tuples) a odeslána v jedné operaci. Data z MariaDB jsou čtena proudově (streaming) po menších dávkách, čímž se zamezuje načtení celého objemu dat do paměti najednou. Veškeré řídicí a logovací tabulky (`ETL_IncrementalControl`, `ETL_RunLog`) jsou spravovány přímo v ClickHouse.

- **Krok 2: Ze stagingu do dimenzí a faktů**

`maria_click_bilina_kamery_staging_to_fact.py` skript spouští sérii SQL dotazů. Pro naplnění dimenzí se používá konstrukce `INSERT INTO ... SELECT ... WHERE NOT IN`, která vybere a vloží pouze nové hodnoty. Následně jsou data transformována do faktové tabulky jediným příkazem `INSERT INTO ... SELECT` obsahujícím všechny potřebné `LEFT JOINy` na dimenzní tabulky.



Obrázek 3.5.: UML porovnání ETL scriptů napříč systémy. Zdroj: vlastní zpracování pomocí aplikace PlantUML Editor [13]



Obrázek 3.6.: UML porovnání ETL scriptů napříč systémy. Zdroj: vlastní zpracování pomocí aplikace PlantUML Editor [13]

	stgid	landingid	originaltime	topic	loadddtm	velocity	ilpc	vehclass	sensor	ip	utc	detectiontype
4	127,537,482		2024-06-26 04:00:12.000 +0200	/Bilina/kamery/comea/BI-TP-12	2025-10-26 15:04:33.922 +0100	-1 CZ		0 BI-TP-12	EDA444879		2024-06-25 09:27:20.729 +0200	detector
5	127,537,483		2024-06-26 04:00:12.000 +0200	/Bilina/kamery/comea/BI-TP-O28	2025-10-26 15:04:33.922 +0100	-1 CZ		0 BI-TP-O28	97746D2A69		2024-06-25 09:27:20.951 +0200	detector
6	127,537,484		2024-06-26 04:00:12.000 +0200	/Bilina/kamery/comea/BI-TP-12	2025-10-26 15:04:33.922 +0100	-1 CZ		0 BI-TP-12	1F4C42739F		2024-06-25 09:27:22.031 +0200	detector
7	127,537,485		2024-06-26 04:00:12.000 +0200	/Bilina/kamery/comea/BI-TP-12B	2025-10-26 15:04:33.922 +0100	-1 CZ		0 BI-TP-12B	92A8446D88		2024-06-25 09:27:22.378 +0200	detector
8	127,537,486		2024-06-26 04:00:12.000 +0200	/Bilina/kamery/comea/BI-MO-11	2025-10-26 15:04:33.922 +0100	-1 CZ		0 BI-MO-11	56C35F0585		2024-06-25 09:27:22.937 +0200	detector

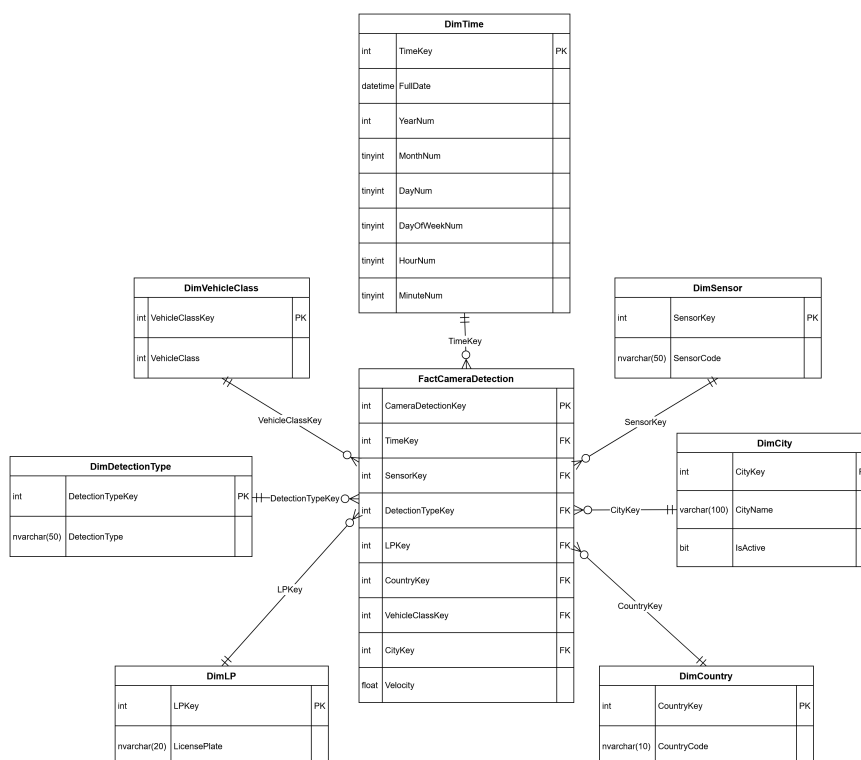
Obrázek 3.7.: Staging tabulka v datovém skladu. Zdroj: vlastní s použitím aplikace DBeaver [22]

Struktura a vrstvy datového skladu

Zdrojová data byla získávána ze systému Portabo, kde jsou ukládána jako MQTT zprávy v JSON formátu. Data mi byla poskytnuta zašifrovaná, abychom je mohli v rámci bakalářské práce využít. Tyto zprávy obsahují informace o detekcích vozidel – například registrační značku, typ detekce, rychlost, město, čas a senzor. Data z vodoměrů, elektroměrů a dalších senzorů. V rámci ETL procesu byla provedena tato fáze:

- **Data lake:** surová data načtená přímo z MariaDB SQLDump,
- **DW Staging:** očištěná, formátovaná a rozparsovaná data,
- **DW Faktová tabulka:** data transformovaná do hvězdicového modelu s vazbami na dimenze.

Každá implementace využívá dávkové zpracování (BATCH_SIZE) a kontrolu posledního zpracovaného záznamu (LastLoadedID), aby bylo možné ETL proces spouštět opakovaně a inkrementálně.



Obrázek 3.8.: Schéma hvězda faktové tabulky a dimenzí pro data z kamerového systému společnosti CAMEA Technology, a. s. na datové platformě Portabo

OLAP a vizualizace

V rámci praktické části byly nad vytvořenými datovými sklady vystavěny analytické vrstvy odpovídající zvolenému technologickému stacku. Pro komerční řešení postavené na platformě Microsoft SQL Server byl využit nativní nástroj SSAS Tabular. Pro open-source databáze (ClickHouse, MariaDB, PostgreSQL) byla nasazena univerzální sémantická vrstva Cube.js. Cílem bylo

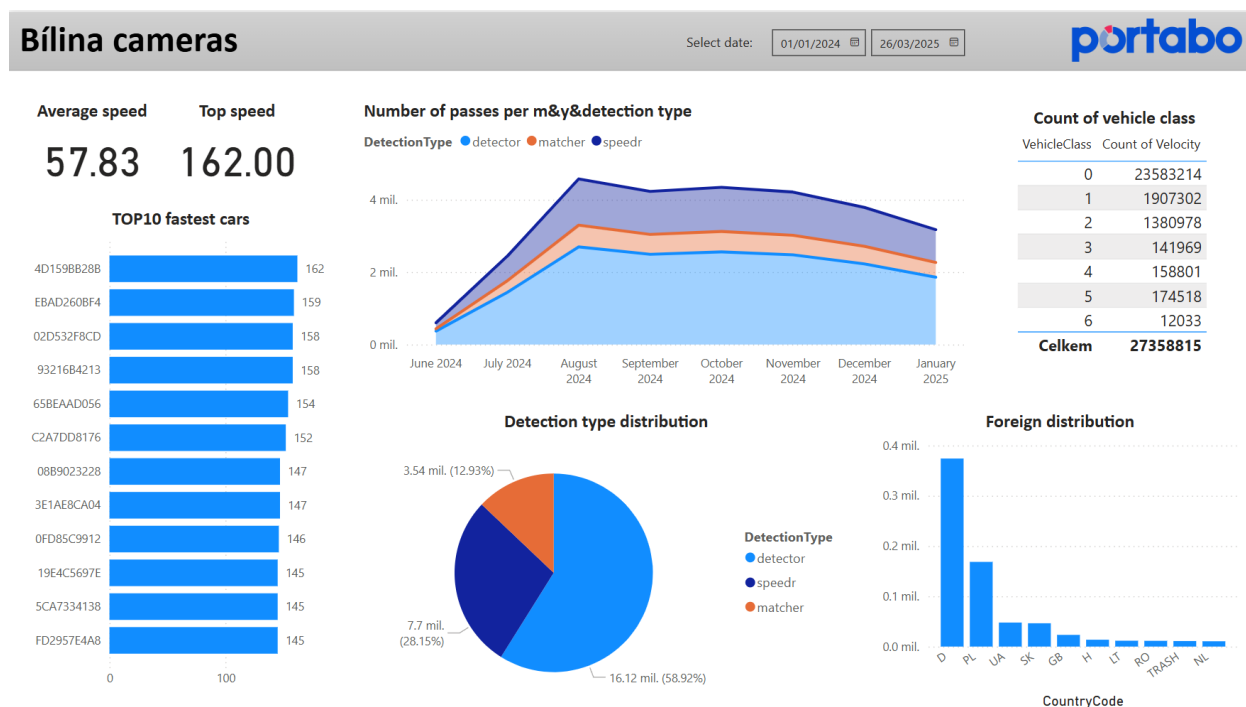
vytvořit funkční OLAP modely umožňující tvorbu interaktivních vizualizací a porovnat rozdíly mezi těmito dvěma přístupy.

Komerční varianta – SSAS Tabular a Power BI

V komerční větvi datové platformy byl vytvořen datový model v prostředí SQL Server Analysis Services (SSAS) Tabular. Model byl vystavěn nad tabulkami FactCameraDetection a dimenzemi DimCity, DimSensor, DimVehicleClass, DimDetectionType, DimLP a DimTime. Propojení tabulek bylo definováno prostřednictvím relačních vazeb podle cizích klíčů, přičemž datový model respektuje hvězdicovou topologii (*star schema*).

Po načtení dat z datového skladu byla v prostředí SSAS vytvořena sada vypočítaných metrik v jazyce DAX. Jednalo se například o:

- Detection Count – celkový počet detekcí,
- Average Velocity – průměrná rychlost vozidel (vylučující nulové hodnoty),
- Unique Plates – počet unikátních registračních značek,
- Detections Over Time – časová agregace počtu detekcí podle dne a hodiny.



Obrázek 3.9.: Ukázka výsledného PBI reportu. Zdroj: vlastní s použitím aplikace Power BI

Součástí modelu byla také implementace základních hierarchií (například *Rok* → *Měsíc* → *Den*) a vybraných filtračních atributů pro přehlednější práci v Power BI. Po ověření funkčnosti byl model nasazen na instanci SSAS Tabular a zpřístupněn uživatelům prostřednictvím Microsoft Power BI a Excel PivotTables. V Power BI byly vytvořeny interaktivní reporty zobrazující přehled detekcí podle města, typu vozidla, senzoru a času. Některé metriky byly navíc doplněny o logiku

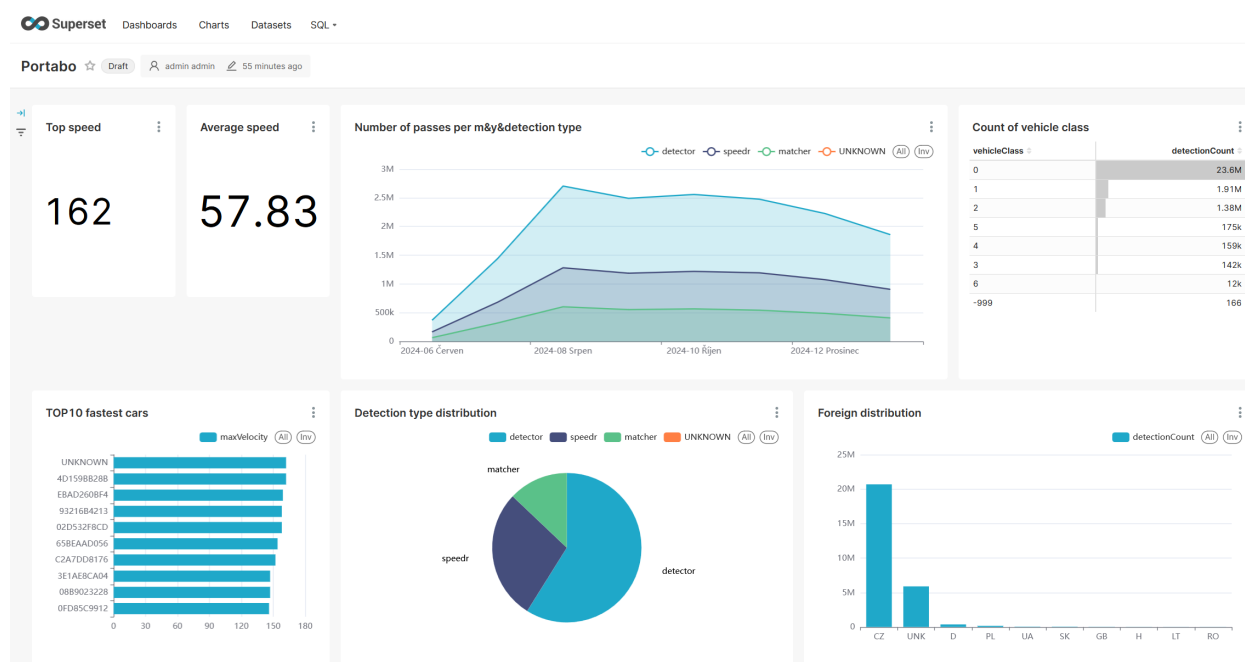
pro kontrolu konzistence dat a filtrování extrémních hodnot. Výhodou tohoto přístupu byla plná integrace s ekosystémem Microsoft, vysoký výkon in-memory výpočtů a možnost implementace bezpečnostních pravidel na úrovni datového modelu.

Open-source varianta – Cube.js a Apache Superset

V open-source větvi datové platformy byla analytická vrstva implementována pomocí nástroje Cube.js, který byl nasazen nad databázemi ClickHouse, MariaDB a PostgreSQL (TimescaleDB). Cube.js v tomto řešení plnil roli mezivrstvy typu OLAP, která sjednocovala přístup k datům z různých databázových systémů a poskytovala standardizované rozhraní pro vizualizaci.

Definice datového modelu byla realizována formou JavaScriptových souborů, přičemž hlavní model byl popsán v souboru `factcameradetection.js`. V tomto souboru byla definována analytická kostka `FactCameraDetection`, která obsahovala popis faktové tabulky a vazby na příslušné dimenze (`DimCity`, `DimSensor`, `DimVehicleClass`, `DimDetectionType`, `DimLP` a `DimTime`). Součástí definice byly také klíčové metriky odpovídající komerční variantě: celkový počet detekcí, průměrná rychlost, minimální a maximální rychlost, počet unikátních SPZ a počet detekcí s nenulovou rychlostí. Model využíval vazby typu `JOIN` na cizí klíče a odpovídal hvězdicovému schématu použitému v datovém skladu.

Po sestavení modelu byl Cube.js spuštěn jako samostatná Node.js služba, která poskytovala rozhraní typu PostgreSQL API. Toto rozhraní umožňuje klientským aplikacím komunikovat s Cube.js stejným způsobem, jako by šlo o běžnou SQL databázi – tedy pomocí SQL dotazů. Díky tomu bylo možné na Cube.js napojit nástroj Apache Superset, který byl použit jako hlavní vizualizační platforma open-source řešení.



Obrázek 3.10.: Ukázka výsledného Superset reportu. Zdroj: vlastní s použitím aplikace Superset

V rámci Superset byl vytvořen interaktivní dashboard zobrazující klíčové metriky: počty detekcí v čase, rozložení typů vozidel, přehled rychlostí. Data byla načítána prostřednictvím SQL dotazů směřovaných na PostgreSQL endpoint poskytovaný Cube.js, čímž byla zajištěna kompatibilita bez nutnosti dalších úprav. Pro zvýšení výkonu byly v Cube.js aktivovány funkce *pre-aggregations* a *query caching*, které umožňovaly ukládat výsledky často opakovaných dotazů.

Porovnání přístupů

Oba přístupy poskytují plnohodnotnou OLAP analytiku, avšak liší se způsobem implementace i správy. SSAS Tabular umožňuje centrální řízení, pokročilé DAX výpočty a přímou integraci s Power BI, což zjednodušuje nasazení ve firemním prostředí. Naopak Cube.js nabízí flexibilitu, otevřenost a možnost úprav datového modelu přímo v kódu, což je výhodné zejména v agilním vývoji nebo při potřebě integrovat analytiku do webové aplikace.

Obě řešení tak umožnila plnohodnotnou vizualizaci a analýzu dat z detekčních systémů, přičemž volba konkrétní technologie závisí především na prostředí, infrastruktuře a požadavcích na rozšiřitelnost systému.

3.3. Provedení srovnávací analýzy

Cílem této kapitoly je objektivní srovnání výkonu a funkčních vlastností implementovaných datových architektur. Analýza byla primárně koncipována jako „**out-of-the-box**“ srovnání, což znamená, že hodnotí výkon systémů v jejich výchozí konfiguraci s běžně používanými indexy. Tento přístup reflektuje realitu organizací, které často nedisponují hlubokou expertizou pro pokročilé ladění databázového jádra.

Srovnání probíhalo ve dvou hlavních rovinách:

1. **Výkon na úrovni databáze:** Přímé měření doby odezvy SQL dotazů na různých databázových systémech a úložných enginech.
2. **Výkon na úrovni BI nástroje:** Hodnocení uživatelského prožitku, konkrétně rychlosti načtení dashboardů.

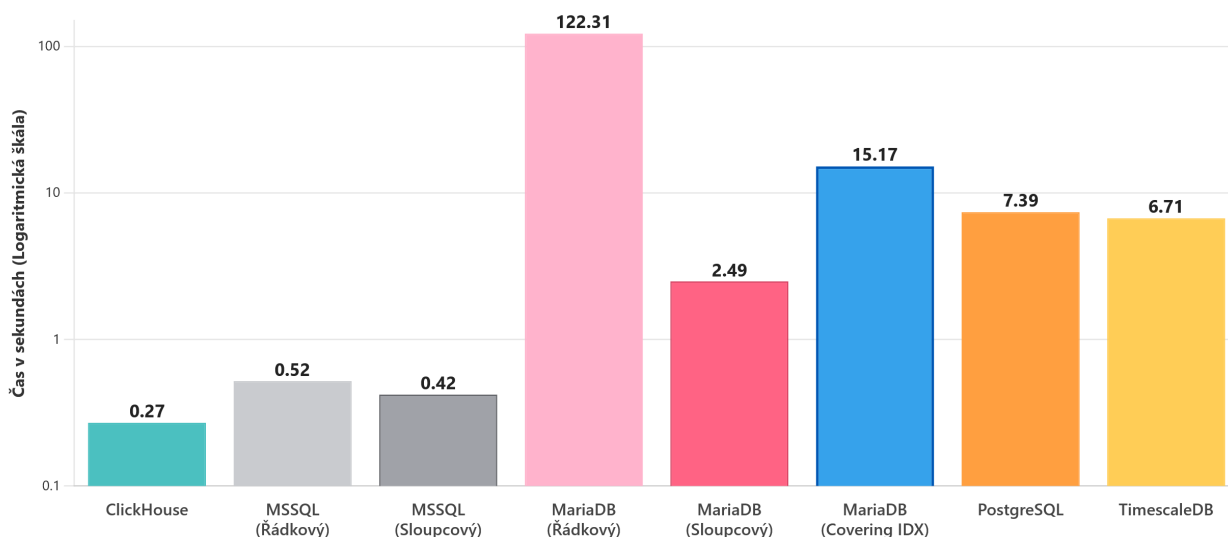
Výkonové parametry

V první fázi byl měřen čas provedení sady pěti reprezentativních analytických dotazů. Cílem bylo porovnat efektivitu tradičního řádkového (row-store) a moderního sloupcového (column-store) úložiště. Každý testovaný dotaz byl spuštěn celkem 100krát, aby se eliminovaly odchylky způsobené externími vlivy. Hodnoty prezentované v následujících grafech představují aritmetický průměr těchto měření. Je důležité upozornit, že následující grafy využívají logaritmickou škálu, což je nutné zohlednit při jejich interpretaci, jelikož rozdíly mezi jednotlivými technologiemi jsou v několika řádech.

Důležitým aspektem srovnání byla také efektivita ukládání dat. Během nahrávání dat (ETL proces) do faktové tabulky se ukázalo, že databáze ClickHouse dosahuje extrémní míry komprese. Oproti ostatním testovaným systémům zabírala data na disku přibližně $10\times$ méně místa. Tato vlastnost, daná pokročilými kompresními algoritmy a sloupcovou orientací, představuje významnou úsporu nákladů na disková úložiště (viz obrázek 3.17) [28].

Detailní znění všech použitých SQL dotazů je k dispozici v příloze na githubu a také v příloze B.

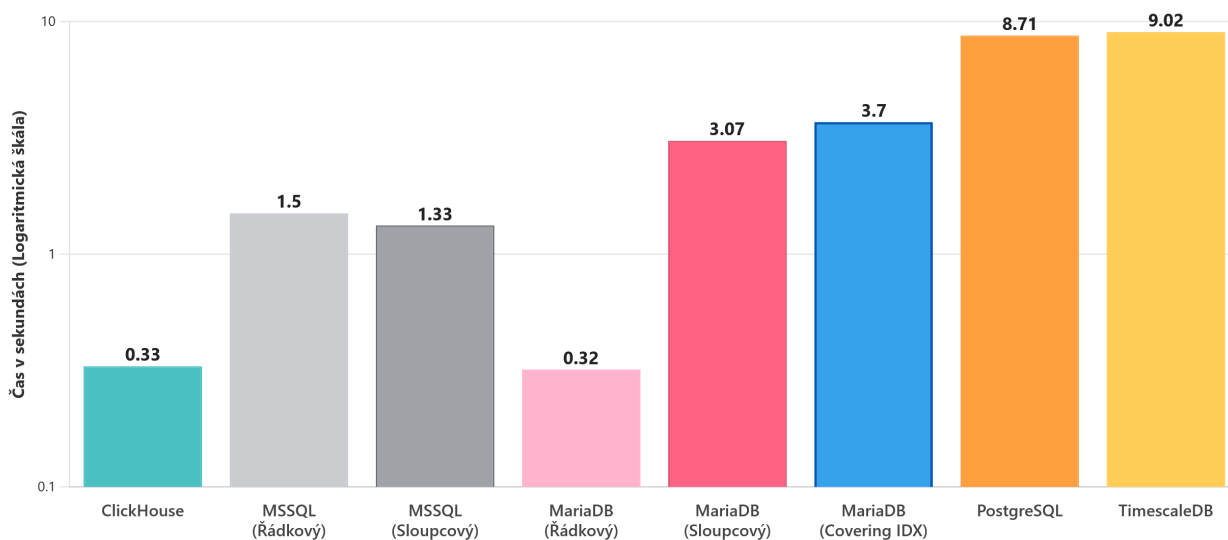
1. Dotaz 1: Jednoduchá agregace s jedním spojením



Obrázek 3.11.: Výkon jednoduché agregace s jedním spojením (Dotaz 1). Méně je lépe. Zdroj: vlastní s využitím knihovny Chart.js [29]

Obrázek 3.11 ukazuje výsledky pro základní analytický dotaz (průměrná rychlost dle třídy vozidla). Zde se naplno projevuje klíčová výhoda sloupcových databází (ClickHouse, MSSQL Col., MariaDB Col.), které načítají pouze relevantní sloupce. Naopak tradiční řádkové úložiště (MariaDB Row) ve výchozím nastavení načítá celé řádky, což při milionech záznamů vede k masivnímu I/O a pomalé odezvě.

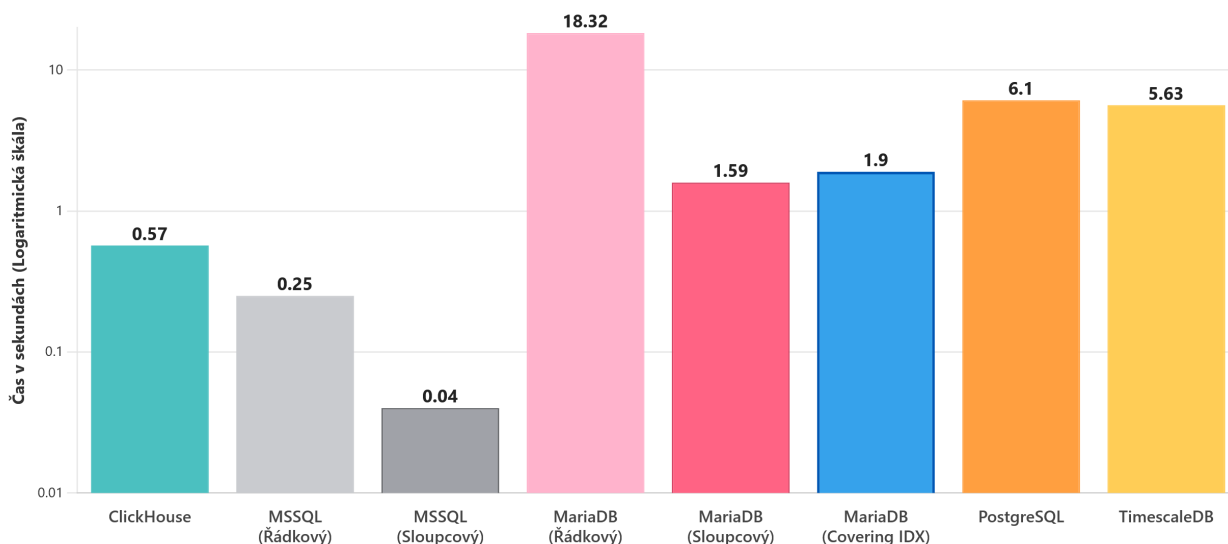
2. Dotaz 2: Agregace nad časovou dimenzí



Obrázek 3.12.: Výkon agregace nad časovou dimenzí (Dotaz 2). Méně je lépe. Zdroj: vlastní s využitím knihovny Chart.js [29]

Druhý dotaz (počet detekcí v čase) odhalil zajímavé chování. Ačkoliv sloupcové databáze dominovaly, řádková MariaDB zde dosáhla překvapivě dobrého výsledku.

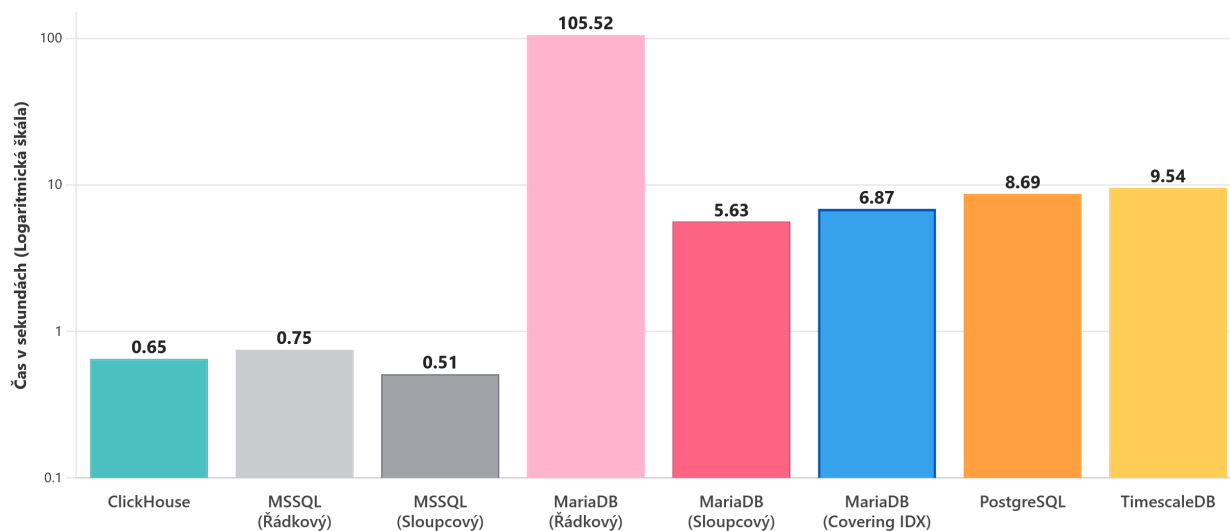
3. Dotaz 3: Vícenásobné spojení a filtrování



Obrázek 3.13.: Výkon dotazu s vícenásobným spojením a filtrováním (Dotaz 3). Méně je lépe. Zdroj: vlastní s využitím knihovny Chart.js [29]

Třetí, komplexnější dotaz, demonstroval sílu optimalizátoru MS SQL.

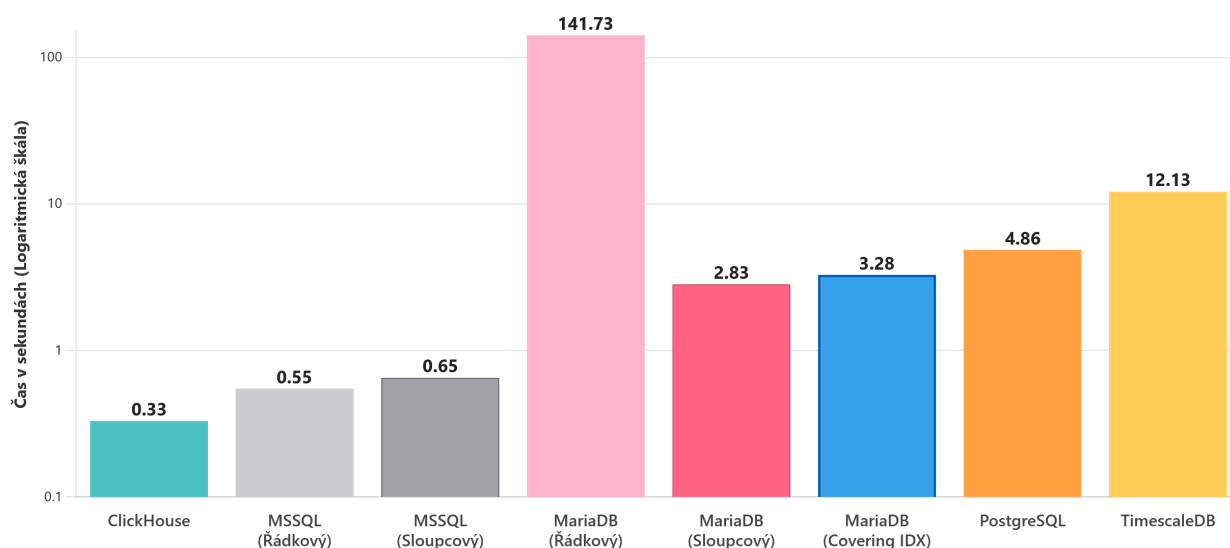
4. Dotaz 4: Agregace nad dvěma dimenzemi



Obrázek 3.14.: Výkon agregace nad dvěma dimenzemi (Dotaz 4). Méně je lépe. Zdroj: vlastní s využitím knihovny Chart.js [29]

U čtvrtého dotazu s vyšší kardinalitou seskupování je opět patrný fatální propad řádkové MariaDB ve srovnání se sloupcovými variantami, které poskytují odezvu v řádu sekund.

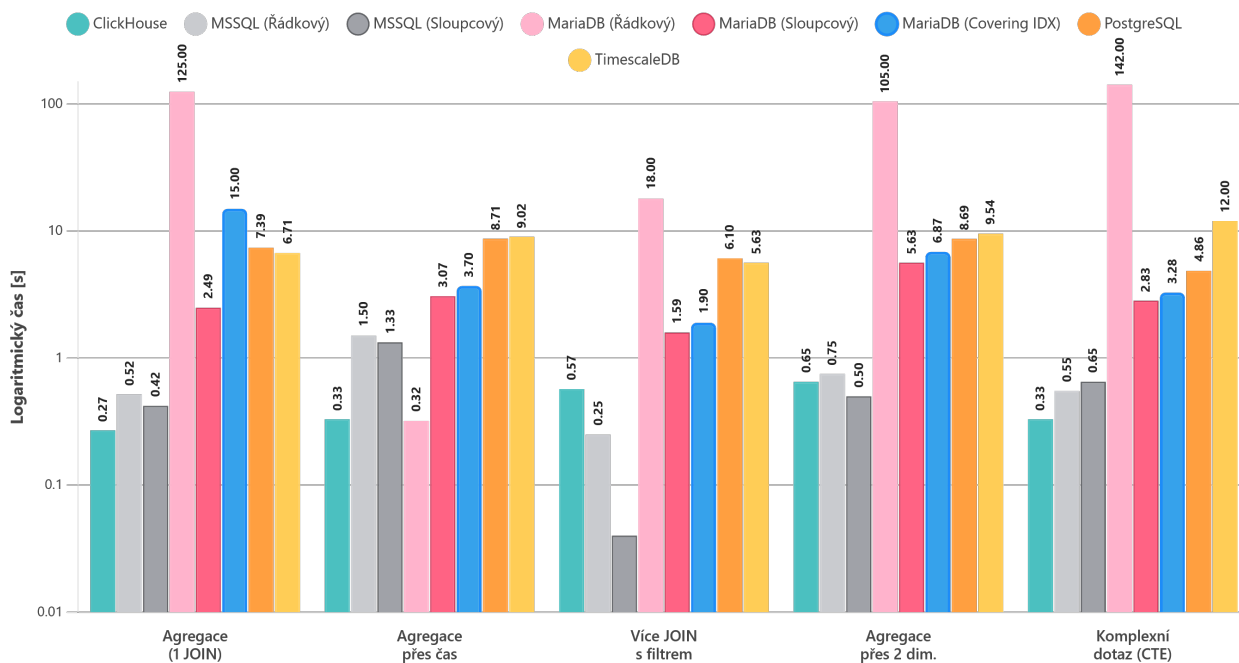
5. Dotaz 5: Komplexní dotaz s CTE a okenní funkcí



Obrázek 3.15.: Výkon komplexního dotazu s CTE a okenní funkcí (Dotaz 5). Méně je lépe. Zdroj: vlastní s využitím knihovny Chart.js [29]

Poslední dotaz (CTE a okenní funkce) potvrdil, že zatímco optimalizátory ClickHouse a MS SQL si s komplexitou poradí, řádková MariaDB v základním nastavení naráží na své limity.

6. Všechny dotazy společně



Obrázek 3.16.: Celkové porovnání výkonu na úrovni databáze. Méně je lépe. Zdroj: vlastní s využitím knihovny Chart.js [29]

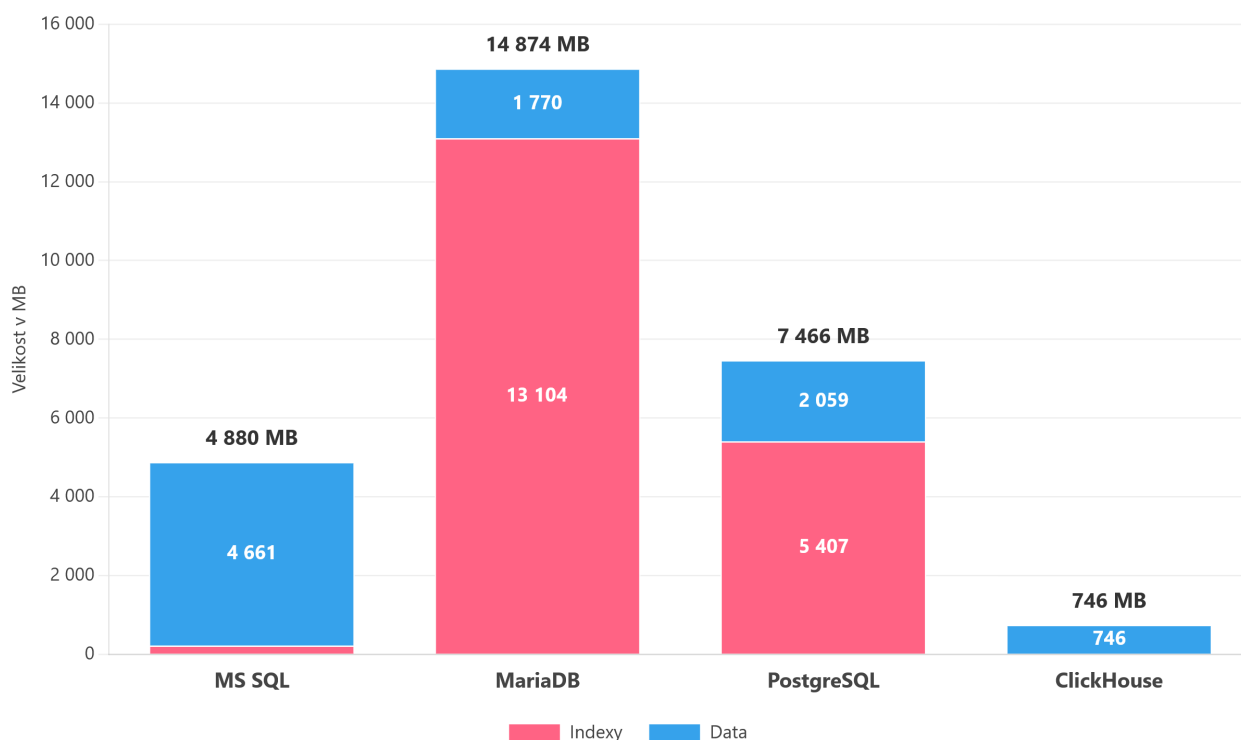
Tabulka 3.1.: Relativní srovnání výkonu (násobek nejrychlejšího času, 1.0x = nejlepší)

Scénář	CH	MS(R)	MS(C)	Ma(R)	Ma(C)	Ma(Cov)	PG	TS
1. Agregace (1 JOIN)	1.0x	1.9x	1.6x	453.0x	9.2x	56.2x	27.4x	24.9x
2. Agregace přes čas	1.03x	4.7x	4.2x	1.0x	9.6x	11.6x	27.2x	28.2x
3. Více JOIN s filtrem	14.3x	6.3x	1.0x	458.0x	39.8x	47.5x	152.5x	140.8x
4. Agregace přes 2 dim.	1.3x	1.5x	1.0x	206.9x	11.0x	13.5x	17.0x	18.7x
5. Komplexní dotaz	1.0x	1.7x	2.0x	429.5x	8.6x	9.9x	14.7x	36.8x

Legenda: CH = ClickHouse, MS = MSSQL, Ma = MariaDB, PG = PostgreSQL, TS = TimescaleDB, (R) = Řádkový, (C) =

Sloupcový, (Cov) = Covering Index

7. Fragmentace dat napříč databázovými systémy



Obrázek 3.17.: Porovnání velikosti dat faktové tabulky na disku mezi jednotlivými databázovými systémy. Zdroj: vlastní s využitím knihovny Chart.js [29]

Vliv strategie indexace na výkon MariaDB (InnoDB)

Vzhledem k výrazným rozdílům ve výkonu řádkové MariaDB (InnoDB) oproti ostatním systémům (PostgreSQL, MS SQL), které i v řádkovém režimu dosahovaly lepších časů, byla provedena dodatečná analýza indexace.

Ukázalo se, že zatímco konkurenční databáze dokázaly efektivně zpracovat analytické dotazy se standardními indexy, MariaDB vyžaduje pro tento typ zátěže odlišný přístup. Řešením byla implementace tzv. **složených pokrývajících indexů** (covering indexes) [30]. Tyto indexy musí explicitně obsahovat nejen klíče pro spojení ('JOIN'), ale i samotná data pro agregaci ('SELECT' klauzule). Tím je vynucena strategie *Using index*, kdy databáze čte data přímo z indexové struktury a zcela eliminuje nákladné čtení z datových souborů na disku.

Obrázek 3.16 ukazuje, jaký dopad měla optimalizace pokrývajících indexů na výkon MariaDB (InnoDB). Po aplikaci těchto specifických indexů došlo k řádovému zrychlení: Tento experiment potvrzuje, že MariaDB je schopna sloužit i pro analytické dotazy, vyžaduje však – na rozdíl od jiných databází – striktní a manuální optimalizaci indexů na míru konkrétním dotazům.

Testování souběžné zátěže (Concurrency)

Pro ověření chování databázových systémů pod zátěží byl realizován test souběžného přístupu (Concurrency Benchmark). Cílem tohoto testu bylo simulovat reálný provoz, kdy k analytickému systému přistupuje více uživatelů současně.

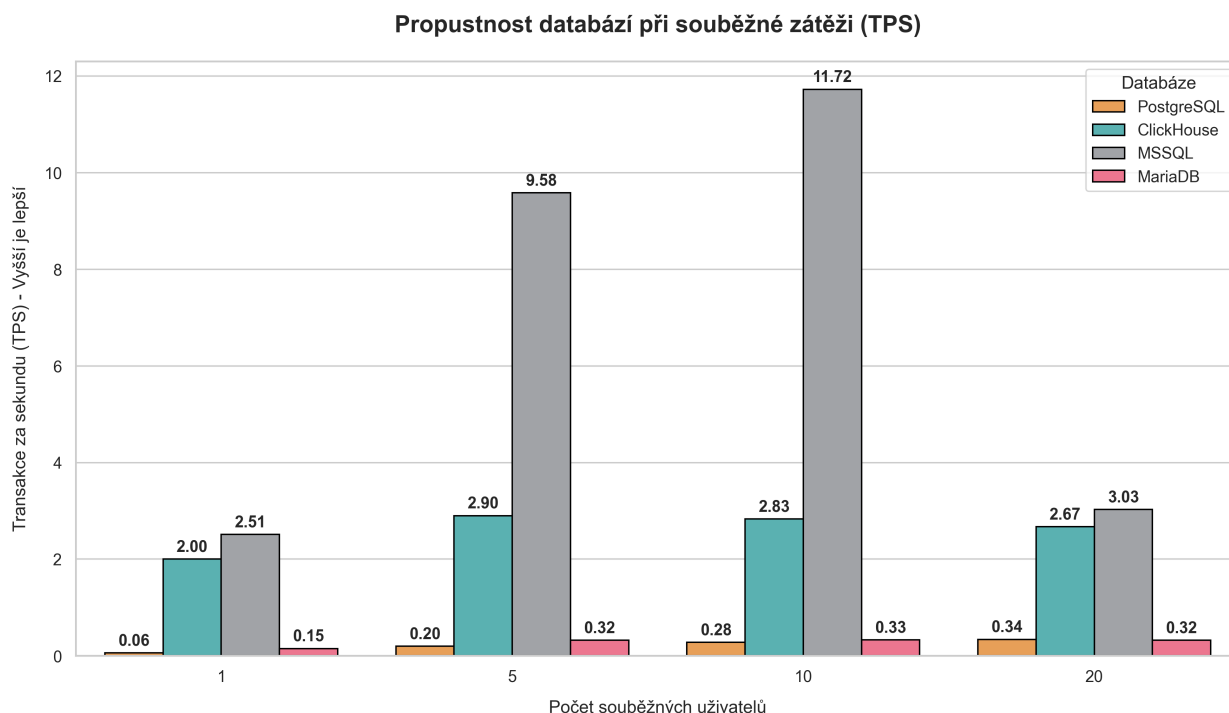
Na rozdíl od předchozích výkonostních testů, které srovnávaly různé varianty úložných enginů (řádkové vs. sloupcové), byly pro tento zátěžový test vybrány pouze *nejefektivnější konfigurace* každé databázové platformy. Cílem bylo porovnat maximální potenciál jednotlivých technologií v jejich optimálním nastavení pro analytickou zátěž (např. MariaDB s enginem ColumnStore, MSSQL s Columnstore indexy).

Testovací scénář byl definován následovně:

- **Zátěž:** Simulace 1, 5, 10 a 20 souběžných virtuálních uživatelů.
- **Úloha:** Každý uživatel cyklicky spouštěl sadu 5 definovaných analytických dotazů (viz předchozí kapitola).
- **Metriky:**
 - **TPS (Transactions Per Second):** Počet úspěšně dokončených dotazů za sekundu. Vyšší hodnota indikuje lepší propustnost.
 - **Avg Latency:** Průměrná doba odezvy jednoho dotazu v sekundách. Nižší hodnota znamená rychlejší odezvu pro uživatele.

Z technického hlediska byl test realizován pomocí vlastního skriptu v jazyce Python s využitím knihovny `concurrent.futures`. Pro simulaci souběžných uživatelů byl nasazen `ThreadPoolExecutor`, který pro každého virtuálního uživatele alokoval samostatné vlákno. Všechna vlákna (např. 20 pro nejvyšší úroveň zátěže) přistupovala k databázi paralelně a v reálném čase odesílala své sady dotazů. Tímto způsobem bylo dosaženo věrné simulace situace, kdy více uživatelů v jeden okamžik vyvolá aktualizaci dashboardu, což nutí databázový stroj zpracovávat desítky náročných analytických dotazů naráz.

Výsledky měření jsou vizualizovány na grafu 3.18.



Obrázek 3.18.: Propustnost databází (TPS) v závislosti na počtu souběžných uživatelů. Vyšší je lepší. Zdroj: vlastní s využitím knihovny Chart.js [29]

Poznámka: Varianta MariaDB s enginem InnoDB byla z tohoto srovnání vyřazena, jelikož časy zpracování byly řádově vyšší a provedení kompletního benchmarku by bylo časově neúměrné.

Interpretace výsledků databázové vrstvy

Souhrnná analýza demonstruje několik klíčových zjištění:

- **Efektivita optimalizátorů dotazů:** Ačkoliv jsou sloupcové databáze (ClickHouse, MariaDB ColumnStore) pro OLAP zátěž přirozeně vhodnější, testy ukázaly, že robustní optimalizátor dotazů (MS SQL Server) dokáže i v řádkovém režimu dosáhnout srovnatelných výsledků.
- **Specifika MariaDB:** Jak bylo prokázáno v doplňkovém testu, řádková MariaDB může být výkonná, ale vyžaduje pokročilou správu indexů (covering indexes), což zvyšuje nároky na údržbu oproti systémům, které jsou výkonné „out-of-the-box“.
- **Výkon nativních OLAP databází:** ClickHouse potvrdil svou dominanci v oblasti analytického zpracování a ve většině testů dosahoval nejlepších výsledků s časy pod jednu sekundu.
- **Univerzálnost MS SQL Serveru:** Komerční řešení od Microsoftu prokázalo vysokou stabilitu a výkon. A to jak při použití Columnstore indexů, kde se blížilo výkonu ClickHouse, tak i v klasickém řádkovém režimu, kde díky pokročilé optimalizaci výrazně překonalo ostatní řádkové databáze.
- **Specifika TimescaleDB:** Výkon TimescaleDB byl v tomto srovnání podobný jako u PostgreSQL. Důvodem je, že TimescaleDB není architektonicky cílena na hvězdicové schéma

(Star Schema). Nejvyšší efektivitu dosahuje při použití denormalizovaných tabulek, kde jsou data uložena v rámci jedné velké struktury (hypertable). Tento přístup umožňuje lépe využít její optimalizační mechanismy, jako je komprese a automatický partitioning.

Zhodnocení souběžné zátěže (Concurrency):

- **ClickHouse:** Tento systém potvrdil svou dominanci pro analytické úlohy s vysokou zátěží. Vykazuje velmi stabilní výkon; i při nárůstu počtu uživatelů se hodnota TPS drží na vysoké úrovni (kolem 2.8) a latence roste pouze lineárně. To z něj činí ideálního kandidáta pro backend analytických dashboardů s více uživateli.
- **MS SQL Server:** Při nižší a střední zátěži (do 10 uživatelů) dosahuje MS SQL Server excellentních výsledků, dokonce výrazně překonávajících ClickHouse (až 11.72 TPS). To svědčí o špičkové optimalizaci pro paralelní zpracování dotazů. Nicméně při vysoké zátěži (20 uživatelů) dochází k prudkému propadu výkonu (na 3.03 TPS) a nárůstu latence. Tento zlom může indikovat vyčerpání systémových prostředků (např. worker threads) nebo zvýšenou režii na zamykání a kontextové přepínání.
- **PostgreSQL a MariaDB:** Oba systémy v tomto testu výrazně zaostávají za sloupcovými databázemi. Hodnoty TPS se pohybují hluboko pod hranicí 1.0, což znamená, že systém nestíhá odbavovat ani jeden dotaz za sekundu. Při 20 uživateli narůstá průměrná latence na desítky sekund (50 s pro PostgreSQL, 60 s pro MariaDB), což je pro interaktivní práci nepřijatelné. Tento výsledek potvrzuje, že pro OLAP zátěž nad velkými daty nejsou tyto systémy ve své základní konfiguraci vhodné a vyžadovaly by buď agresivní optimalizaci (pre-agregace), nebo jiný architektonický přístup.

Funkční vlastnosti

Práce s mapovými daty

Součástí zadání byla i analýza práce s geoprostorovými daty. Oba testované nástroje nabízejí možnosti vizualizace na mapě, avšak s rozdílným přístupem.

- **Power BI (Azure Maps / Bing Maps):** Vizualizační vrstva Power BI je postavena na mapových službách Azure Maps a Bing Maps ¹. Z technického hlediska je klíčovou výhodou *aktivní geokódovací služba*, která na pozadí automaticky převádí textové atributy (názvy měst, adresy, PSČ) na geografické souřadnice. To umožňuje vizualizovat data, která neobsahují explicitní GPS souřadnice. Tato abstrakce však přináší výkonnostní limity a závislost na externím API, což může být při velkém objemu dotazů (tisíce bodů) pomalé nebo zpoplatněné. Vykreslování bodů probíhá převážně na straně klienta, což omezuje plynulost při vyšších počtech objektů.
- **Apache Superset (Deck.gl):** Pro geoprostorovou vizualizaci využívá Superset knihovnu *Deck.gl* (WebGL) ², která umožňuje GPU akcelerované vykreslování milionů bodů v reálném

¹<https://learn.microsoft.com/en-us/azure/azure-maps/power-bi-visual-get-started>

²<https://superset.apache.org/docs/configuration/country-map-tools>

čase. Zásadním rozdílem je, že Superset *nemá integrovanou geokódovací službu* pro převod adres na souřadnice. Pro bodové mapy (Scatterplot, Arc, Heatmap) vyžaduje, aby data v databázi již obsahovala explicitní sloupce pro zeměpisnou šířku a délku (latitude/longitude). Výjimkou jsou tzv. *Country Maps*, které dokáží mapovat data na oblasti (kraje, státy) na základě ISO kódů (ISO 3166-2) ³, avšak pro detailní bodovou vizualizaci je nutná příprava dat na úrovni ETL procesu (např. využití PostGIS).

Je však důležité zmínit, že absenci nativního geokódování lze efektivně řešit na úrovni datové přípravy. V rámci platformy Portabo je tento nedostatek kompenzován využitím vlastních převodnickových tabulek, které obsahují předpřipravené GeoJSON geometrie pro různé administrativní celky. Systém tak disponuje mapovými podklady pro obce, obce s rozšířenou působností (ORP), okresy, kraje, regiony soudržnosti (NUTS2), ale i pro detailnější jednotky, jako jsou základní sídelní jednotky (ZSJ), katastrální území či adresní body. Tyto geometrie vycházejí z dat Registru územní identifikace, adres a nemovitostí (RÚIAN) a umožňují vizualizaci dat na mapě i na základě běžných identifikátorů, jako je PSČ nebo kód obce.

Srovnání vizualizačních možností a architektury reportingu

Srovnání vizualizační vrstvy se nezaměřuje pouze na estetickou stránku, ale především na technickou architekturu vykreslování, způsob generování dotazů a flexibilitu vývojového cyklu.

- **Microsoft Power BI (Canvas-based & Model-first):** Architektura reportů je založena na principu plátna (canvas), kde jsou vizuální komponenty umísťovány v absolutních pozicích.
 - **Architektura vykreslování (Client-side):** Většina nativních vizuálů je vykreslována v prohlížeči pomocí HTML5 Canvas a knihoven postavených na D3.js. Power BI uplatňuje limity na počet renderovaných datových bodů (tzv. *Data Point Limits*, např. cca 3 500 bodů pro bodový graf) ⁴. Při práci s rozsáhlými datasety používá techniky redukce dat (*High Density Sampling*), které mohou snížit přesnost vykreslených extrémů ⁵.
 - **Layout engine:** Umožňuje přesné umístění prvků a jejich vrstvení (tzv. pixel-perfect design). Nevýhodou je nutnost vytvářet samostatné rozložení pro mobilní zařízení.
 - **Interaktivita (DAX Context):** Interakce uživatele (např. kliknutí do vizuálu) mění tzv. *Filter Context*. Engine následně automaticky generuje odpovídající DAX dotazy pro všechny ovlivněné vizuály na stránce. Správné fungování vyžaduje korektně definované relace v datovém modelu.
 - **Rozšiřitelnost:** Ekosystém je uzavřený, avšak rozšiřitelný pomocí *Custom Visuals* SDK (TypeScript/React) ⁶.
- **Apache Superset (Grid-based & SQL-first):** Dashboards využívají mřížkový systém rozložení (Grid system) s modulárními komponentami.

³<https://superset.apache.org/docs/configuration/country-map-tools>

⁴<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-high-density-scatter-charts>

⁵<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-high-density-scatter-charts>

⁶<https://learn.microsoft.com/en-us/power-bi/developer/visuals/>

- **Architektura vykreslování (Query-based):** Superset funguje jako tenká vrstva nad databázemi a datovými enginy, které provádějí veškeré výpočty a agregace ⁷. Každá vizualizace je založena na SQL dotazu, jehož výsledná data jsou backendem serializována do formátu JSON a odeslána frontendu. Frontend poté vykresluje grafy pomocí otevřených vizualizačních knihoven, zejména *deck.gl* (pro mapové a geovizuální grafy využívající WebGL akceleraci) ⁸. Díky WebGL dokáže *deck.gl* efektivně zobrazovat i velké množství geografických nebo bodových objektů, což je zásadní pro práci s rozsáhlejšími geodaty.
- **Layout engine:** Komponenty jsou organizovány do řádků a sloupců, což poskytuje nativní responzivitu. Tento přístup však omezuje volné pozicování prvků oproti canvas-based nástrojům.
- **Interaktivita (Event-driven):** Cross-filtering je implementován pomocí událostí publikovaných do globálního stavu dashboardu. Tyto události spouštějí nové SQL dotazy pro všechny vizuály zahrnuté ve *scoping* pravidlech. Podobně jako v Power BI je cross-filtering plně podporován a umožňuje interaktivní filtrování dashboardu kliknutím na jednotlivé vizuály.
- **SQL Lab:** Superset obsahuje vestavěné SQL IDE umožňující uživatelům psát vlastní dotazy a vytvářet vizualizace přímo z výsledku. Tento *SQL-first* přístup zrychluje explorativní analýzu a nevyžaduje sémantický model předem.

Uživatelská přívětivost

Z hlediska uživatelské přívětivosti lze konstatovat, že oba testované systémy dosahují srovnatelné úrovně, ačkoliv každý cílí na mírně odlišný typ uživatele. Komerční řešení (Power BI) vyniká možností tvorby *pixel-perfect* layoutů grafů a hlubokou integrací s kancelářskými nástroji, což ocení zejména běžní uživatelé a management. Open-source varianta (Apache Superset) naopak nabízí větší volnost a flexibilitu, kterou preferují technicky zdatnější uživatelé a datoví analytici vyžadující maximální kontrolu.

Obecně lze říci, že oba nástroje představují vyzrálá a profesionální řešení, která lze doporučit pro nasazení v podnikovém prostředí. Finální volba by tak neměla být otázkou kvality uživatelského rozhraní, která je u obou variant vysoká, ale spíše otázkou specifických preferencí, technického zázemí a licenční strategie dané organizace.

Ekonomické aspekty a TCO

Vzhledem k tomu, že praktická část této práce ověřovala široké spektrum technologií – od tradičních relačních databází (PostgreSQL, MariaDB) přes specializované analytické systémy (Clic-

⁷<https://superset.apache.org/docs/>

⁸<https://superset.apache.org/docs/configuration/country-map-tools>

kHouse, MariaDB ColumnStore) až po komplexní komerční ekosystémy (MS SQL Server + SSAS) – je nutné náklady hodnotit v širším kontextu.

Celkové náklady na vlastnictví (Total Cost of Ownership – TCO) [31] v tomto případě dělíme na tři hlavní složky:

- **Licenční a infrastrukturní náklady** (cloud, HW, software),
- **Náklady na lidský kapitál** (mzdy a know-how),
- **Náklady na implementaci** (časová dotace vývoje).

Náklady na implementaci a lidský kapitál

Při srovnání nákladů na lidskou práci je nutné zohlednit nejen přímou hodinovou sazbu, ale i dostupnost specialistů na trhu a křivku učení.

- **Srovnatelná pracnost ETL (Python):** Jelikož byla pro oba přístupy (komerční i open-source) zvolena strategie ETL s využitím jazyka Python, objem práce nutný pro napsání datových pump byl v podstatě totožný. Vývoj skriptů pro pyodbc (MSSQL) vyžadoval srovnatelnou časovou investici jako implementace clickhouse-driver [32] či pyscopg2 [27].
- **Vliv křivky učení:** Vyšší časová náročnost implementace open-source analytických řešení byla způsobena primárně nutností osvojit si specifické know-how (např. konfigurace MariaDB ColumnStore, optimalizace pre-agregací v Cube.js), které není na trhu tak běžně dostupné jako znalost ekosystému Microsoft (SSAS/SSMS).
- **Dostupnost expertízy:** Zatímco administrátora pro PostgreSQL lze na trhu práce nalézt relativně snadno, specialista na sloupcové databáze (ColumnStore, ClickHouse) je profil výrazně vzácnější, což se odráží v ceně práce.

Pro kvantifikaci těchto nákladů byly využity průměrné měsíční hrubé mzdy v IT sektoru v České republice pro rok 2024. Data vycházejí z kombinace statistik ČSÚ a platových průzkumů personálních agentur (např. Hays, Grafton)⁹. Rozptyl mezd zohledňuje senioritu a regionální rozdíly.

Tabulka 3.2.: Odhad měsíčních nákladů na lidský kapitál dle technologií a rolí

Technologie (Systém)	Typová role	Odhad mzdy (CZK)
PostgreSQL, MariaDB	Database Administrator (Standard)	70 000 – 90 000
MariaDB ColumnStore, ClickHouse	Big Data / DWH Engineer	90 000 – 120 000
MS SQL Server + SSAS	BI Developer (MS Stack)	80 000 – 110 000
Cube.js, Superset	Data Analyst / Frontend	55 000 – 75 000
Python (ETL procesy)	Python Developer / Data Engineer	80 000 – 100 000

Z tabulky 3.2 vyplývá, že ačkoliv open-source řešení šetří náklady na licencích, může generovat vyšší nároky na mzdové ohodnocení specialistů schopných tyto systémy efektivně spravovat a integrovat.

⁹Hays Czech Republic: Salary Guide 2024. Dostupné z veřejných reportů pro IT sektor.

Licenční a infrastrukturní náročnost open-source ekosystému

V rámci open-source řešení (např. kombinace ClickHouse a Apache Superset) lze identifikovat specifika, která zásadně snižují celkové náklady na vlastnictví (TCO).

- **Licenční politika:** Využití licencí typu Apache 2.0 nebo MIT eliminuje přímé licenční poplatky. To umožňuje neomezené horizontální škálování infrastruktury (přidávání serverů) bez nárůstu administrativních či licenčních nákladů, což je v kontrastu s komerčními modely vázanými na počet jader (Per Core).
- **Hardwarová efektivita:** Sloupcové databáze (např. ClickHouse) využívají efektivní kompresi a streamování dat z disku. Nevyžadují, aby se celý datový model nacházel v operační paměti (RAM), což umožňuje dosahovat vysokého výkonu i na komoditním hardwaru s nižší kapacitou paměti.

Licenční a infrastrukturní náročnost komerčního řešení (Microsoft Stack)

U referenční architektury založené na technologiích Microsoft (SQL Server, SSAS, Power BI) tvoří licenční poplatky dominantní složku TCO. Organizace o velikosti 50 uživatelů stojí před volbou mezi dvěma licenčními modely: plně On-Premise (vyžaduje vyšší edici serveru) nebo Hybridní (vyžaduje poplatky za uživatele).

- **SQL Server – Licenční model a limity:** Pro nasazení ve firemním prostředí se obvykle volí model *Per Core*, který je nezbytný pro externí přístup.
 - **SQL Server Standard:** Tato edice je cenově dostupnější (cca 3 500–4 000 USD za 2 jádra). Maximální využitelná paměť pro analytickou službu SSAS (*Tabular*) je *limitována na 64 GB RAM* na instanci (od verze 2022) [33]. To je sice hardwarově dostačující pro střední modely, ale tato edice neumožňuje provozovat on-premise Power BI Report Server.
 - **SQL Server Enterprise:** Pro modely překračující 64 GB RAM nebo pro nárok na *Power BI Report Server* je nutná edice Enterprise. Cena této edice je přibližně čtyřnásobná oproti verzi Standard (cca 14 000 USD a více za 2 jádra).
- **Infrastrukturní dopad (In-Memory daň):** Technologie SSAS Tabular funguje na principu in-memory. U velkých datasetů tak organizace naráží na tvrdý limit edice Standard (64 GB). Překročení tohoto limitu nebo požadavek na on-premise vizualizaci vynucuje nákup drahé Enterprise licence, což činí vertikální škálování finančně neefektivním.
- **Power BI (Vizualizační vrstva):**
 - **Power BI Pro (Cloud):** Pro hybridní scénář je nutná licence cca 10 USD / uživatel / měsíc. Tato licence je součástí pouze nejvyšších plánů Office 365 (E5), u běžných plánů (E3, Business Standard) je nutné ji dokoupit.

- **Power BI Report Server (On-Premise):** Tento server nelze zakoupit samostatně pro jednotlivé uživatele. Je dostupný pouze jako benefit k licenci SQL Server Enterprise s aktivní Software Assurance (SA) nebo v rámci Premium capacity.

Srovnání nákladů: Následující tabulky porovnávají plně on-premise variantu (vysoký CAPEX) a hybridní variantu (nižší CAPEX, ale trvalý OPEX za uživatele).

Tabulka 3.3.: Varianta A: Plně On-Premise řešení (Power BI Report Server) pro 50 uživatelů

Položka	Typ nákladu	Jednotková cena	Cena pro 50 uživatelů (1. rok)
Hardware (Server)	CAPEX (Jednorázový)	150 000 CZK	150 000 CZK
SQL Server Ent (4 jádra)*	OPEX (Licence + SA)	~ 750 000 CZK	750 000 CZK
Power BI Report Server	Součást Enterprise SA	0 CZK	0 CZK
Celkem (1. rok)			900 000 CZK

*Pro on-premise Power BI je nutná edice Enterprise se Software Assurance (SA).

Tabulka 3.4.: Varianta B: Hybridní řešení (SQL Standard + Power BI Cloud) pro 50 uživatelů

Položka	Typ nákladu	Jednotková cena	Cena pro 50 uživatelů (1. rok)
Hardware (Server)	CAPEX (Jednorázový)	150 000 CZK	150 000 CZK
SQL Server Std (4 jádra)	OPEX (Licence)	~ 170 000 CZK	170 000 CZK
Power BI Pro	OPEX (Roční subs.)	~ 3 000 CZK / uživ.	150 000 CZK
Celkem (1. rok)			470 000 CZK

U hybridního modelu postačuje edice Standard (limit 64 GB pro SSAS 2022).

Hardwarové nároky (MOLAP vs. ROLAP)

Významný rozdíl v TCO tvoří také hardwarové požadavky, které vycházejí z odlišných architektur:

- **MOLAP (MS SSAS):** Vyžaduje, aby se celý datový model (nebo jeho podstatná část) vešel do operační paměti (RAM). RAM je nejdražší komponentou serveru a s růstem dat rostou náklady na ni lineárně až exponenciálně.
- **ROLAP (ClickHouse/MariaDB):** Spoléhá na rychlé diskové operace. Data jsou uložena na discích (SSD/NVMe), které jsou řádově levnější než RAM. Díky efektivní kompresi (zmíněná 10× úspora u ClickHouse) jsou nároky na kapacitu úložiště minimální.

Analýza ukázala, že zatímco pracnost implementace (ETL v Pythonu) je u obou řešení srovnatelná, ekonomické náklady se rozevírají při škálování:

- **Komerční řešení** je ekonomicky výhodné pro menší datové modely (do limitu RAM edice Standard) a uzavřené skupiny uživatelů. Poskytuje komfort integrovaného prostředí.

- **Open-source řešení** (ClickHouse + Cube.js) má zpočátku vyšší nároky na specifickou expertizu, ale s růstem objemu dat se stává výrazně levnějším. Neexistuje zde "skokový" náklad při přechodu na vyšší edici a neplatí se za pasivní konzumenty reportů.

3.4. Zhodnocení výsledků a doporučení

Cílem této práce bylo porovnat vhodnost komerčních a open-source technologií pro analytickou platformu Portabo. Na základě provedených zátěžových testů, analýzy funkcionality a ekonomické rozvahy lze konstatovat, že oba přístupy představují validní řešení, avšak každý je vhodný pro odlišný kontext nasazení.

Zatímco komerční řešení (MS SQL + Power BI) vyniká okamžitou použitelností a výkonem „out-of-the-box“ díky in-memory technologii, open-source varianta (ClickHouse + Cube.js) vítězí v efektivitě ukládání dat a dlouhodobé udržitelnosti nákladů při škálování.

Pro přehlednost jsou klíčové rozdíly mezi dvěma finálními kandidáty shrnuty v následující tabulce. Databáze MariaDB a PostgreSQL byly z finálního srovnání vyloučeny, neboť jejich výkonnostní limity je pro účely OLAP analýzy nad velkými daty diskvalifikují.

Tabulka 3.5.: Finální srovnání architektury pro platformu Portabo

Parametr	MS SQL + SSAS (Komerční)	ClickHouse + Cube.js (Open-source)
Určení	Enterprise reporting, Management	Big Data, IoT telemetrie, Embedded Analytics
Výkon	Extrémní (In-Memory MOLAP), ale limitovaný velikostí RAM.	Vysoký (Columnar ROLAP), lineárně škálovatelný s hardwarem.
Efektivita uložení	Standardní komprese.	<i>Extrémní komprese</i> (až 10× úspora místa oproti PostgreSQL).
Náklady (OPEX)	Vysoké (licence per-core/user). Skokový růst nákladů při přechodu na Enterprise.	Minimální (zdarma licence, levnější HW).
Náklady (CAPEX)	Nízké (out-of-the-box řešení).	Vysoké (know-how, vývoj).
Flexibilita	Vendor lock-in (uzavřený ekosystém).	Vysoká modularita, API-first přístup, možnost vlastních úprav.
Lidské zdroje	Dostupnější na trhu (BI Developer).	Náročnější na expertizu (Senior DB/Backend Engineer).

Závěrečné doporučení pro Portabo

Pro prostředí platformy Portabo, která pracuje s objemnými telemetrickými daty z IoT senzorů a kamerových či jiných systémů, se jako *vhodnější jeví open-source varianta postavená na technologii ClickHouse*.

Toto doporučení vychází z následujících klíčových zjištění praktické části:

1. **Výkon a optimalizace:** Překvapivým zjištěním bylo, že open-source databáze *ClickHouse* dokázala výkonnostně konkurovat, a v některých testech i předčit, komerční in-memory řešení MS SQL Server (SSAS). Tradiční řádkové databáze (např. MariaDB) se ukázaly jako použitelné pouze za předpokladu pokročilé manuální optimalizace (například covering indexes), bez níž byl jejich výkon pro OLAP nedostatečný.
 2. **Efektivita úložiště:** ClickHouse prokázal v testech schopnost ukládat data s přibližně desetinásobnou úsporou diskového prostoru oproti tradičním systémům. Pro telemetrická data, která neustále narůstají, je tato vlastnost kritická.
 3. **Ekonomika škálování a limity architektury:** Srovnání ukázalo zásadní rozdíl v přístupu ke škálování. Zatímco komerční in-memory technologie (SSAS) je technicky vázána na kapacitu operační paměti serveru a licenční model Microsoftu by se při nárůstu objemu dat stal finančně neudržitelným, open-source stack (ClickHouse) efektivně využívá diskové úložiště a umožňuje lineární škálování na komoditním hardwaru bez licenčních poplatků.
 4. **Flexibilita integrace:** Nasazení Cube.js jako sémantické vrstvy umožňuje oddělit datovou logiku od vizualizace. To dává platformě Portabo svobodu v budoucnu změnit vizualizační nástroj bez nutnosti přepisovat datové modely.
- **Komerční řešení (MS SQL + Power BI):** Vyniká okamžitou použitelností a výkonem „out-of-the-box“ díky in-memory technologii.
 - **Open-source varianta (ClickHouse + Cube.js):** Vítězí v efektivitě ukládání dat a dlouhodobé udržitelnosti nákladů při škálování.

4. Závěr

Tato bakalářská práce se komplexně zabývala návrhem, implementací a srovnáním moderních datových architektur pro analytickou platformu Portabo. Hlavním cílem bylo ověřit, zda mohou open-source technologie plnohodnotně konkurovat zavedeným komerčním ekosystémům v náročné oblasti zpracování IoT telemetrie.

Výstupy práce: implementační vzory

Hmatatelným výstupem práce je sada ověřených implementačních vzorů, které slouží jako referenční architektura pro budoucí rozvoj platformy:

- **ETL procesy:** Byla vytvořena sada optimalizovaných Python skriptů pro extrakci a transformaci dat (detailně popsáno v sekci 3.2).
- **Datové sklady (DWH):** Práce poskytuje srovnání a implementační schémata pro čtyři různé databázové enginy (MS SQL, MariaDB, PostgreSQL, ClickHouse), včetně optimalizačních technik (indexace, komprese) (viz sekce 3.2).
- **Sémantická vrstva:** Byly navrženy a ověřeny dva přístupy k definici datových modelů – tradiční SSAS Tabular (Microsoft) a moderní „Headless BI“ přístup pomocí Cube.js (viz sekce 3.2).
- **Vizualizace:** Práce demonstrovala integraci s Power BI i open-source nástrojem Apache Superset.

Shrnutí srovnání a klíčová zjištění

V rámci praktické části bylo provedeno detailní srovnání v několika rovinách:

- **Výkonnostní srovnání:** Testy prokázaly, že sloupcová databáze ClickHouse dokáže v analytických dotazech (OLAP) nejen konkurovat, ale v mnoha případech i překonat in-memory technologii MS SQL Server Analysis Services (SSAS) (výsledky testů viz sekce 3.3). To vyvrací mýtus, že pouze drahá in-memory řešení jsou vhodná pro rychlou interaktivní analýzu.

- **Ekonomická rozvaha (TCO):** Analýza nákladů odhalila zásadní rozdíl v modelu financování. Komerční řešení (Microsoft Stack) nabízí nízký vstupní práh (CAPEX) díky „out-of-the-box“ funkcionalitě, ale trpí vysokými provozními náklady (OPEX) při škálování (licence per-core, limity RAM). Naopak open-source cesta vyžaduje vyšší počáteční investici do know-how a vývoje (CAPEX), ale zajišťuje minimální provozní náklady a udržitelnost i při exponenciálním růstu dat (rozbor TCO viz sekce 3.3).
- **Architektonická flexibilita:** Zatímco komerční stack uživatele uzamyká v proprietárním ekosystému (Vendor Lock-in), navržená open-source architektura (ClickHouse + Cube.js) je modulární. Umožňuje nezávisle měnit vizualizační nástroje nebo databázový backend bez nutnosti přepisovat celou logiku (viz sekce 3.2).

Závěrečné doporučení

Na základě všech zjištění práce jednoznačně doporučuje pro platformu Portabo přechod na open-source architekturu postavenou na databázi **ClickHouse** a případně sémantické vrstvě **Cube.js**. Toto řešení nabízí nejlepší poměr ceny a výkonu, eliminuje licenční rizika a poskytuje robustní základ pro zpracování Big Data, který je pro IoT platformu klíčový.

Seznam použitých zdrojů

1. BRAGIN, Tanya. *The Unbundling of the Cloud Data Warehouse* [online]. ClickHouse Inc., 2023-11. [cit. 2025-10-23]. Dostupné z: <https://clickhouse.com/blog/the-unbundling-of-the-cloud-data-warehouse>.
2. PRIEBE, Torsten; NEUMAIER, Sebastian; MARKUS, Stefan. Von Data Warehouse bis Data Mesh: Ein Wegweiser durch den Dschungel analytischer Datenarchitekturen. *arXiv preprint arXiv:2212.03612* [online]. 2022 [cit. 2025-10-23]. Dostupné z: <https://arxiv.org/abs/2212.03612>.
3. CUBE.DEV TEAM. *Semantic Layer and AI: The Future of Data Querying with Natural Language* [online]. Cube.dev, 2024-08. [cit. 2025-10-23]. Dostupné z: <https://cube.dev/blog/semantic-layer-and-ai-the-future-of-data-querying-with-natural-language>.
4. SHARMA, Ayush. *Introduction to Cube.js – The Semantic Layer for Building Data Apps* [online]. 2023-02. [cit. 2025-10-23]. Dostupné z: https://heyayush.com/post/2023-02-22_cube-js.
5. GEBRIM, Josue Luzardo. *Cube: Creating a Semantic Data Layer!* [online]. Medium, 2023-03. [cit. 2025-10-23]. Dostupné z: <https://medium.com/codex/cube-creating-a-semantic-data-layer-a947fd0c11f6>.
6. OPEN SOURCE INITIATIVE. *The Open Source Definition* [online]. 2024. [cit. 2025-11-22]. Dostupné z: <https://opensource.org/osd>.
7. IBM. *On-premises vs. Cloud: What's the difference?* [online]. 2024. [cit. 2025-11-22]. Dostupné z: <https://www.ibm.com/topics/on-premises-vs-cloud>.
8. MICROSOFT. *Data warehousing in Microsoft Azure* [online]. Microsoft Learn, 2024. [cit. 2025-12-02]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/example-scenario/data/data-warehouse>.
9. PRAJAPATI, Jignesh. *Implementing Data Products on Google Data Lakehouse Architecture* [online]. Medium, 2023. [cit. 2025-12-02]. Dostupné z: <https://medium.com/google-cloud/implementing-data-products-on-google-data-lakehouse-architecture-df5c5f4cc07e>.
10. AMAZON WEB SERVICES. *Amazon Redshift system overview* [online]. AWS Documentation, 2024. [cit. 2025-12-02]. Dostupné z: https://docs.aws.amazon.com/redshift/latest/dg/c_high_level_system_architecture.html.

11. JUNIOR, Juarez. *Oracle Autonomous Data Warehouse Cloud Service (ADW) - Part 3* [online]. Medium, 2019. [cit. 2025-12-02]. Dostupné z: <https://juarezjunior.medium.com/oracle-autonomous-data-warehouse-cloud-service-adw-part-3-getting-started-with-oracle-machine-1b72224b4112>.
12. MERMAID JS. *Mermaid Live Editor* [online]. Mermaid JS, 2024. [cit. 2025-12-02]. Dostupné z: <https://mermaid.live/>.
13. PLANTUML. *PlantUML: Open-source tool for drawing UML diagrams* [online]. PlantUML, 2024. [cit. 2025-12-02]. Dostupné z: <https://plantuml.com/>.
14. AMAZON WEB SERVICES; TALEND. *Data Warehouse Modernization: From Monolith to Modular* [online]. Amazon Web Services, 2019. [cit. 2025-10-23]. Dostupné z: https://pages.awscloud.com/rs/112-TZM-766/images/GLOBAL_PTNR_STPM_DWM_Talend_eBook_Aug-2019.pdf.
15. DATABRICKS. *The Modern Data Stack: How the Evolution of Data Architecture Led to the Data Intelligence Platform* [online]. Databricks, 2024. [cit. 2025-10-23]. Dostupné z: <https://www.databricks.com/blog/modern-data-stack-how-evolution-data-architecture-led-data-intelligence-platform>.
16. ANTHROPIC. *Model Context Protocol* [online]. 2024. [cit. 2025-11-30]. Dostupné z: <https://modelcontextprotocol.io/>.
17. INMON, William H. *Building the Data Warehouse*. 4. vyd. John Wiley & Sons, 2005. ISBN 978-0-764-59944-6.
18. KIMBALL, Ralph; ROSS, Margy. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3. vyd. John Wiley & Sons, 2013. ISBN 978-1-118-53080-1.
19. WIKIPEDIA CONTRIBUTORS. *Star schema* [online]. Wikipedia, 2024. [cit. 2025-12-02]. Dostupné z: https://en.wikipedia.org/wiki/Star_schema.
20. WIKIPEDIA CONTRIBUTORS. *Snowflake schema* [online]. Wikipedia, 2024. [cit. 2025-12-02]. Dostupné z: https://en.wikipedia.org/wiki/Snowflake_schema.
21. FONTAINE, Dimitri. *pgloader: Migrate to PostgreSQL in a single command* [online]. 2024. Ver. 3.6.9 [cit. 2025-12-02]. Dostupné z: <https://github.com/dimitri/pgloader>.
22. DBEAVER CORP. *DBeaver: Free Universal Database Tool* [online]. 2024. [cit. 2025-12-02]. Dostupné z: <https://dbeaver.io/>.
23. BRAY, Tim. *The JavaScript Object Notation (JSON) Data Interchange Format* [Internet Requests for Comments]. RFC Editor, 2017-12. RFC, 8259. Internet Engineering Task Force (IETF). Dostupné také z: <https://www.rfc-editor.org/rfc/rfc8259.txt>. Dostupné online.
24. LESKOVEC, Jure; RAJARAMAN, Anand; ULLMAN, Jeffrey D. *Mining of Massive Datasets*. 3. vyd. Cambridge University Press, 2020. ISBN 978-1108476348. Dostupné také z: <http://www.mmms.org/>. Kapitola 3: Finding Similar Items.

25. KLEEHAMMER, Michael. *pyodbc: Python ODBC Bridge* [online]. 2024. [cit. 2025-12-02]. Dostupné z: <https://github.com/mkleehammer/pyodbc>.
26. MATSUBARA, Yutaka. *PyMySQL: Pure Python MySQL Client* [online]. 2024. [cit. 2025-12-02]. Dostupné z: <https://github.com/PyMySQL/PyMySQL>.
27. DI GREGORIO, Federico; VARRAZZO, Daniele. *Psycopg: PostgreSQL database adapter for the Python programming language* [online]. 2024. [cit. 2025-12-02]. Dostupné z: <https://www.psycopg.org/>.
28. CLICKHOUSE, INC. *Data Compression in ClickHouse* [online]. 2024. [cit. 2025-11-30]. Dostupné z: <https://clickhouse.com/docs/en/sql-reference/statements/alter/compression>.
29. CHART.JS CONTRIBUTORS. *Chart.js: Simple yet flexible JavaScript charting for designers & developers* [online]. 2024. [cit. 2025-12-04]. Dostupné z: <https://www.chartjs.org/>.
30. MARIADB FOUNDATION. *Optimization and Indexes: Covering Index* [online]. 2024. [cit. 2025-11-30]. Dostupné z: <https://mariadb.com/kb/en/optimization-and-indexes/>.
31. GARTNER, INC. *Total Cost of Ownership (TCO)* [online]. 2024. [cit. 2025-11-30]. Dostupné z: <https://www.gartner.com/en/information-technology/glossary/total-cost-of-ownership-tco>.
32. LEBEDEV, Konstantin. *clickhouse-driver: Python driver for ClickHouse* [online]. 2024. [cit. 2025-12-02]. Dostupné z: <https://github.com/mymarilyn/clickhouse-driver>.
33. MICROSOFT. *Editions and supported features of SQL Server 2022* [online]. 2024. [cit. 2025-11-30]. Dostupné z: <https://learn.microsoft.com/en-us/sql/sql-server/editions-and-components/sql-server-2022/editions-and-supported-features-of-sql-server-2022>.

A. Externí přílohy

Externí přílohy této bakalářské práce, včetně kompletních zdrojových kódů, skriptů a dokumentace, jsou dostupné ve veřejném repozitáři na službě GitHub:

https://github.com/ladislav-tahal/KI_UJEP_THESIS_LADISLAV_TAHAL

Struktura přiloženého elektronického nosiče (a repozitáře) je následující:

```
/
|-- Analýza témat/ ..... Analýza MQTT témat a zpráv
|   |-- Topik100.xlsx ..... Témata se 100% shodou payloadu
|   '-- Topik50.xlsx ..... Témata s částečnou shodou
|-- Architektura/ ..... Grafické návrhy řešení
|   '-- reporting_structure.png ..... Schéma architektury reportingu
|-- Bakalářská práce PDF/ ..... Text práce v jazyce LaTeX
|
|-- MSSQL_MSSQL_SSAS_PowerBI/ ..... Komerční větev (MS SQL Server)
|   |-- OlapTabular/ ..... Projekt SSAS Tabular modelu
|   '-- Portabo - kamery Bílina.pbix ..... Power BI report
|-- MariaDB_Clickhouse_CubeJS_Superset/ .... Open-source větev (ClickHouse)
|   |-- cubejs/ ..... Datové modely pro Cube.js
|   |-- superset/ ..... Konfigurace Apache Superset
|   |-- docker-compose.yml ..... Orchestrace kontejnerů
|   '-- clickhouse_data.tar.gz ..... Export datového schématu a dat
|-- MariaDB_MariaDB_CubeJS_Superset/ ..... Varianta s MariaDB ColumnStore
|   |-- cubejs/ ..... Datové modely
|   |-- sample_data.sql ..... Generování testovacích dat
|   |-- Dockerfile ..... Konfigurace prostředí
|   '-- docker-compose.yml ..... Orchestrace kontejnerů
|-- PostgreSQL_PostgreSQL_CubeJS_Superset/ . Varianta s PostgreSQL
|   |-- cubejs/ ..... Datové modely
|   '-- docker-compose.yml ..... Orchestrace kontejnerů
|-- Scripty/ ..... Pomocné nástroje a skripty
|   |-- Python/ ..... ETL skripty a generátory dat
|   |-- SQL/ ..... SQL skripty pro inicializaci
|   '-- Start sluzeb/ ..... Skripty pro spouštění služeb
'-- Zdroje/ ..... Seznam použitých zdrojů
```


B. Testovací SQL dotazy

V této příloze jsou uvedeny SQL dotazy použité pro výkonnostní testování databáze ClickHouse. Dotazy pro ostatní testované systémy (MSSQL, MariaDB, PostgreSQL) jsou analogické, s drobnými úpravami respektujícími specifika daného SQL dialektu (např. syntaxe pro práci s datem nebo limitování počtu řádků). Kompletní sada dotazů pro všechny platformy je k dispozici v příloženém repozitáři.

Dotaz 1: Jednoduchá agregace

Listing B.1: Agregace průměrné rychlosti dle třídy vozidla

```
SELECT vc.VehicleClass, avg(f.Velocity) AS AverageVelocity
FROM FactCameraDetection f
JOIN DimVehicleClass vc ON f.VehicleClassKey = vc.VehicleClassKey
GROUP BY vc.VehicleClass;
```

Dotaz 2: Agregace přes časovou dimenzi

Listing B.2: Počet detekcí v čase

```
SELECT t.FullDate, count(*) AS NumberOfDetections
FROM FactCameraDetection f
JOIN DimTime t ON f.TimeKey = t.TimeKey
GROUP BY t.FullDate
ORDER BY t.FullDate;
```

Dotaz 3: Vícenásobné spojení a filtrování

Listing B.3: Top 10 měst s nejvyšší průměrnou rychlostí (filtr na CZ a osobní auta)

```
SELECT c.CityName, avg(f.Velocity) AS AverageVelocity
FROM FactCameraDetection f
JOIN DimCity c ON f.CityKey = c.CityKey
JOIN DimCountry co ON f.CountryKey = co.CountryKey
JOIN DimVehicleClass vc ON f.VehicleClassKey = vc.VehicleClassKey
WHERE co.CountryCode = 'CZ' AND vc.VehicleClass = '2'
GROUP BY c.CityName
ORDER BY AverageVelocity DESC
LIMIT 10;
```

Dotaz 4: Agregace nad dvěma dimenzemi

Listing B.4: Průměrná rychlost dle typu detekce a třídy vozidla

```
SELECT dt.DetectionType, vc.VehicleClass, avg(f.Velocity) AS
    AverageVelocity
FROM FactCameraDetection f
JOIN DimDetectionType dt ON f.DetectionTypeKey = dt.DetectionTypeKey
JOIN DimVehicleClass vc ON f.VehicleClassKey = vc.VehicleClassKey
GROUP BY dt.DetectionType, vc.VehicleClass
ORDER BY dt.DetectionType, vc.VehicleClass;
```

Dotaz 5: Komplexní dotaz (CTE a Window functions)

Listing B.5: Analýza víkendového provozu s využitím CTE a okenních funkcí

```
WITH WeekendDetections AS (  
    SELECT f.CityKey, f.CountryKey, f.VehicleClassKey, f.Velocity  
    FROM FactCameraDetection f  
    JOIN DimTime t ON f.TimeKey = t.TimeKey  
    WHERE toDayOfWeek(t.FullDate) IN (6, 7)  
)  
SELECT co.CountryCode, c.CityName, vc.VehicleClass,  
    avg(wd.Velocity) AS AverageVelocity,  
    dense_rank() OVER(PARTITION BY co.CountryCode ORDER BY count(*)  
        DESC) AS CityRankByDetections  
FROM WeekendDetections wd  
JOIN DimCity c ON wd.CityKey = c.CityKey  
JOIN DimCountry co ON wd.CountryKey = co.CountryKey  
JOIN DimVehicleClass vc ON wd.VehicleClassKey = vc.VehicleClassKey  
GROUP BY co.CountryCode, c.CityName, vc.VehicleClass  
ORDER BY co.CountryCode, CityRankByDetections;
```


Seznam použitých zkratek a pojmů

API Application Programming Interface. Rozhraní pro programování aplikací, umožňující komunikaci mezi softwarovými komponentami.

BI Business Intelligence. Sada procesů, architektur a technologií pro převod surových dat na smysluplné informace pro podporu rozhodování.

CPU Central Processing Unit. Centrální procesorová jednotka, hlavní výpočetní prvek počítače.

CSV Comma-Separated Values. Jednoduchý textový formát pro výměnu tabulkových dat.

DAX Data Analysis Expressions. Jazyk vzorců a dotazů používaný v Power BI a SSAS.

DBMS Database Management System (SŘBD). Software pro správu databází.

DWH Data Warehouse (Datový sklad). Centrální úložiště dat integrovaných z různých zdrojů, optimalizované pro analýzu.

ELT Extract, Load, Transform. Moderní přístup k integraci dat, kdy transformace probíhá až v cílovém datovém skladu.

ETL Extract, Transform, Load. Proces extrakce dat ze zdrojů, jejich transformace a následné nahrání do cílového systému.

GDPR General Data Protection Regulation. Nařízení EU o ochraně osobních údajů.

HTML Hypertext Markup Language. Značkovací jazyk pro tvorbu webových stránek.

IaaS Infrastructure as a Service. Cloudový model poskytující virtualizované výpočetní zdroje přes internet.

IDE Integrated Development Environment. Integrované vývojové prostředí.

IoT Internet of Things. Síť fyzických zařízení vybavených senzory a softwarem pro výměnu dat.

ISO International Organization for Standardization. Mezinárodní organizace pro normalizaci.

JSON JavaScript Object Notation. Odlehčený formát pro výměnu dat, snadno čitelný pro lidi i stroje.

Licence Právní nástroj upravující práva a povinnosti při užití díla. Permisivní licence od Apache Software Foundation, umožňující komerční využití a modifikaci. . General Public License, copyleftová licence vyžadující, aby odvozená díla byla šířena pod stejnou licencí. . Permisivní softwarová licence vytvořená Massachusetts Institute of Technology, umožňující volné užití kódu s minimálními omezeními. .

LLM Large Language Model. Typ modelu umělé inteligence trénovaný na obrovském množství textu.

MCP Model Context Protocol. Standard pro propojení AI modelů s externími daty a nástroji.

MOLAP Multidimensional Online Analytical Processing. Technologie ukládající data ve vícerozměrných polích (kostkách) optimalizovaných pro rychlé dotazování.

MPP Massively Parallel Processing. Architektura pro paralelní zpracování dat na mnoha uzlech.

MQTT Message Queuing Telemetry Transport. Lehký síťový protokol pro přenos zpráv mezi zařízeními, často využívaný v IoT.

NLQ Natural Language Querying. Dotazování se na data pomocí přirozeného jazyka.

OLAP Online Analytical Processing. Technologie pro rychlé provádění složitých analytických dotazů nad vícerozměrnými daty.

PaaS Platform as a Service. Cloudový model poskytující vývojářům platformu pro tvorbu aplikací.

RAG Retrieval-Augmented Generation. Technika pro zlepšení přesnosti generativních AI modelů pomocí externích dat.

RAM Random Access Memory. Operační paměť počítače s náhodným přístupem.

RDBMS Relational Database Management System. Systém řízení báze dat založený na relačním modelu.

REST Representational State Transfer. Architektonický styl pro návrh síťových aplikací.

ROLAP Relational Online Analytical Processing. Forma OLAP, která provádí dynamické analýzy přímo nad daty uloženými v relační databázi.

SaaS Software as a Service. Model poskytování softwaru jako služby přes internet.

SDK Software Development Kit. Sada nástrojů pro vývoj softwaru.

SQL Structured Query Language. Standardizovaný dotazovací jazyk pro práci s relačními databázemi.

SSAS SQL Server Analysis Services. Technologie společnosti Microsoft pro online analytické zpracování (OLAP) a dolování dat.

SSD Solid State Drive. Polovodičový disk bez pohyblivých částí, vyznačující se vysokou rychlostí čtení a zápisu.

SSIS SQL Server Integration Services. Platforma pro integraci dat a transformace (ETL) od společnosti Microsoft.

TCO Total Cost of Ownership. Celkové náklady na vlastnictví, zahrnující pořizovací i provozní náklady.

XML Extensible Markup Language. Značkovací jazyk pro ukládání a přenos dat.